
✅ **1. Introduction to Java**

Java is **platform-independent**, **object-oriented**, **strongly typed**, **compiled + interpreted**.
Bytecode is generated after compilation → runs on **JVM**.
Java supports **Automatic Garbage Collection**.

✅ **2. System.out.println() Breakdown**

System.out.println("Hello");

- System: final class from java.lang, contains static fields.
- out: static reference variable of type PrintStream.
- println(): method of PrintStream class.

🧠 MCQ Tip:
System is a **class**, out is a **static reference variable**, and println() is a **method**.

✅ **3. Comments in Java**

1 > // Single-line comment

2> /*
Multi-line comment
*/

3> /**
* JavaDoc comment for documentation generation
*/

🧠 MCQ Tip: Java has 3 types of comments.

✅ **4. Data Types in Java**

➤ **Primitive Types (8 total)**

Type	Size	Default Value
byte	1 byte	0
short	2 bytes	0
int	4 bytes	0
long	8 bytes	0L
float	4 bytes	0.0f
double	8 bytes	0.0d
char	2 bytes	'\u0000'
boolean	1 bit	false

➤ **Reference Types**

Classes (String, Scanner, etc.), Arrays (int[], String[]), Interfaces
Default value: null

✅ **5. Default Values (Important for MCQs)**

```
class Test {
   static int i;
   static float f;
   static boolean b;
   static String s;
   public static void main(String[] args) {
      System.out.println(i); // 0
      System.out.println(f); // 0.0
      System.out.println(b); // false
      System.out.println(s); // null
   }
}
```

🧠 If not initialized:

Primitives → zero/false equivalents.

Reference types → null.

🧠 For **local variables**, you **must initialize before use**.

```
void method() {
   int x;
   System.out.println(x); // Compile-time error
}
```

✅ **6. float vs double Initialization**

```
float a = 12.5;   // ❌ Error: needs 'f' suffix
float a = 12.5f;  // ✅

double d = 12.5;  // ✅
```

🧠 Default decimal literal is double.

✅ **7. String vs String[]**

```
String name = "Java";      // Reference to string object
String[] arr = new String[3]; // Reference to string array
```

🧠 String is a **reference type**, **immutable**, stored in String pool (if literal).

✅ **8. Java is Pass-by-Value**

Even objects are **passed by value**, but the value is a reference.

✅ **JVM (Java Virtual Machine) – Quick Notes**

🔹 **What is JVM?**

- JVM = Java Virtual Machine, part of JRE.
- Responsible for running Java .class files (bytecode).
- Ensures **WORA** (Write Once, Run Anywhere).
- Invokes main() method at runtime.

🧠 **JVM Architecture Components:**

1️⃣ **Class Loader Subsystem**

Handles:

- **Loading** – Reads .class files → stores metadata in **Method Area**.
  - Creates a Class object in Heap (used via getClass()).
- **Linking**
  - **Verification** – Bytecode validity (VerifyError on failure).
  - **Preparation** – Allocates memory for static vars with default values.
  - **Resolution** – Converts symbolic references to actual memory references.
- **Initialization** – Assign values to static vars and run static blocks (top-down & parent-child order).

📌 **Types of Class Loaders:**

Loader Type	Loads from	Implementation
Bootstrap Loader	JAVA_HOME/lib	Native (C/C++)
Extension Loader	JAVA_HOME/jre/lib/ext or java.ext.dirs	Java (ExtClassLoader)
System/Application	CLASSPATH (project classes)	Java (AppClassLoader)

Delegation Principle:

System → Extension → Bootstrap → If not found, ClassNotFoundException.

🧪 Example:

```
System.out.println(String.class.getClassLoader()); // null (bootstrap)
System.out.println(Test.class.getClassLoader());   // AppClassLoader
```

Extras

String.class

- This means: "Give me the Class object that represents the String class."
- Every class in Java has an associated Class<?> object in the JVM.
- So String.class is an instance of Class<String>.

.getClassLoader()

- This is a method on the Class object.
- It returns the ClassLoader that loaded that class**.**

ℹ️ **Example:**

System.out.println(String.class.getClassLoader());

- String is a core Java class, part of the Java standard library (java.lang.String).
- It is loaded by the Bootstrap ClassLoader.

- But the Bootstrap ClassLoader is not a Java object—it is implemented in native code.
- So in Java, calling .getClassLoader() on a class loaded by the bootstrap class loader returns null.

🧠 **That's why you see: null**

✅ **Now for the second line:**
System.out.println(Test.class.getClassLoader());

- Test is your own class (custom code).
- It is usually loaded by the Application ClassLoader (also called AppClassLoader).
- That loader is a proper Java object, so it returns:

sun.misc.Launcher$AppClassLoader@...

2️⃣ **JVM Memory Areas**

Memory Area	Description
Method Area	Class-level data: names, methods, static vars. Shared by all threads.
Heap Area	Stores all objects. Shared by all threads.
Stack Area	Each thread has its own stack (stores method calls, local variables).
PC Register	Each thread has one; stores current instruction address.
Native Method Stack	Stores native method info per thread.

3️⃣ **Execution Engine**
Executes bytecode from .class file using:

- **Interpreter** – Reads and executes bytecode **line by line**.
- **JIT Compiler** – Converts bytecode to **native code** for faster repeated execution.
- **Garbage Collector (GC)** – Destroys unused objects to free memory.

4️⃣ **JNI (Java Native Interface)**

- Provides interaction between **Java code** and **native libraries** (C/C++).
- Enables Java to call or be called by native code.

5️⃣ **Native Method Libraries**

- Libraries written in C/C++ used via JNI for platform-specific operations.

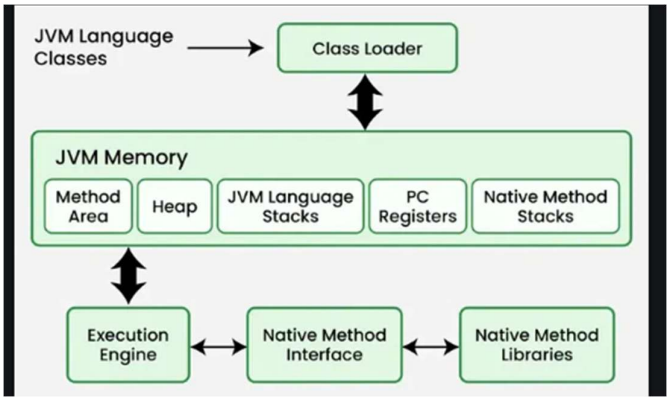
✅ **Sample Output Example**

```
Student s1 = new Student();
Class c1 = s1.getClass();
System.out.println(c1.getName()); // Student
System.out.println(c1 == s1.getClass()); // true
```

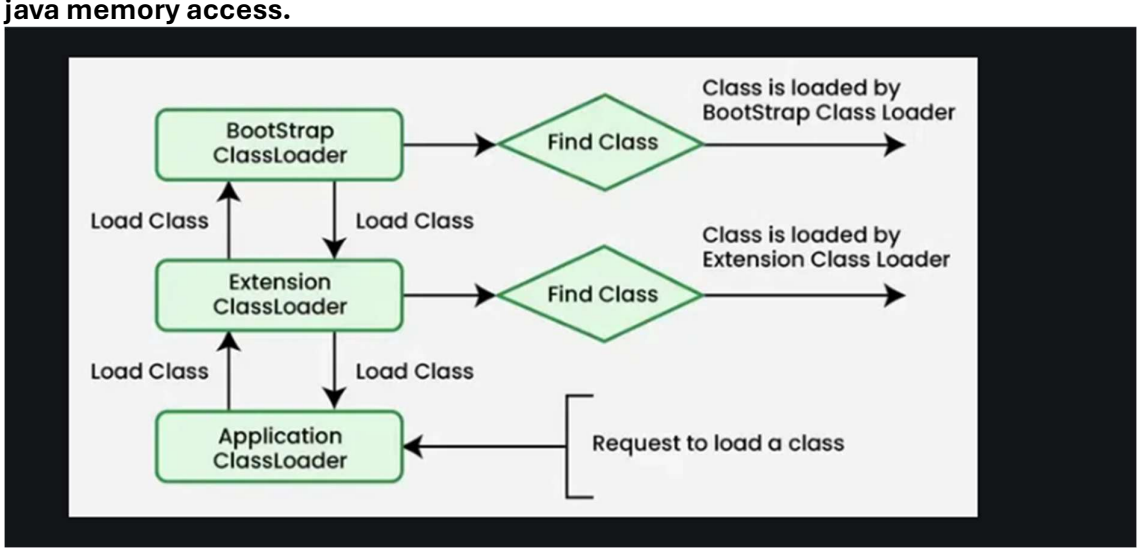
🧠 **Important Points for MCQs:**

- .class file contains **bytecode**.
- JVM is part of **JRE**, not JDK.
- Each class is loaded once → only one Class object per .class.
- Default class loader for user code = **Application (System) Loader**.
- **VerifyError** occurs if class verification fails.
- **JIT** improves interpreter performance.
- String.class.getClassLoader() → null (because loaded by bootstrap loader).
- JVM is responsible for **memory management**, **class loading**, and **execution**.

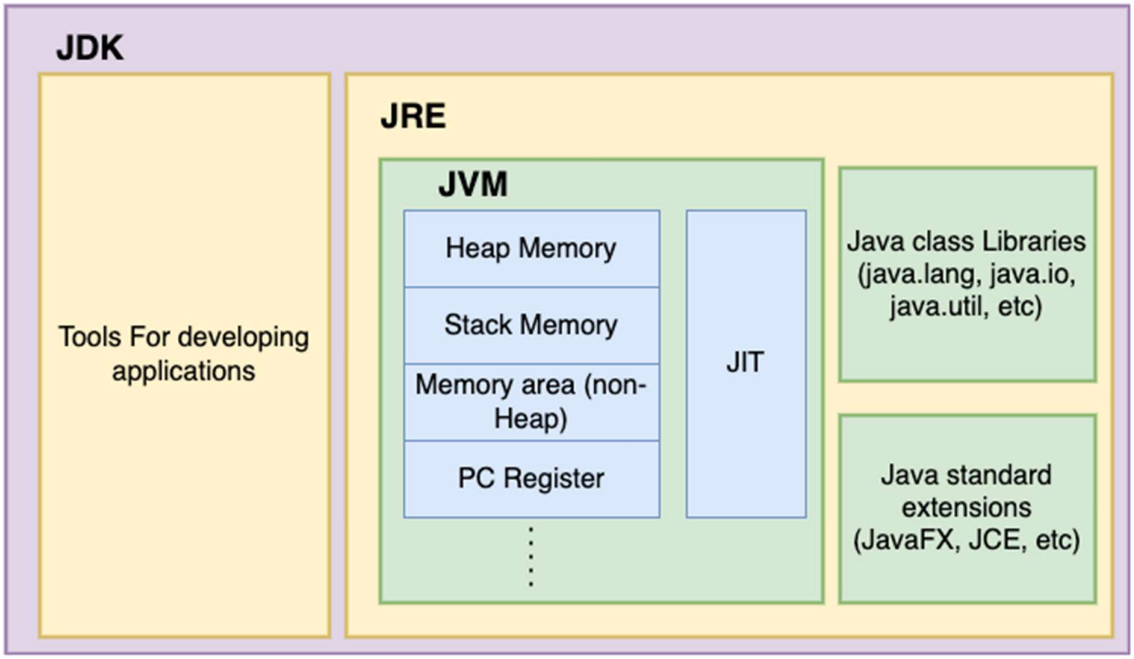
Jvm Architecture



java memory access.



JDK >>JRE >>JVM



Class & Objects

✅ **What is a Class in Java?**

- A **class** is a **blueprint** or **template** for creating objects.
- It defines **fields (variables)** and **methods (behaviors)**.

🔹 **Syntax:**

```
class Student {
  int id;         // field (variable)
  String name;
  void display() {   // method
    System.out.println(id + " " + name);
  }
}
```

✅ **What is an Object?**

- An **object** is a **real instance** of a class.
- It has **state (variables)** and **behavior (methods)**.

🔹 **Syntax:**

```
Student s1 = new Student(); // object creation
```

- new keyword → allocates memory in **Heap**
- Student() → calls constructor
- s1 → reference variable pointing to object

✅ **How Class and Object Relate**

- Class → abstract concept (e.g., *Car*)
- Object → concrete instance (e.g., *Honda City*)

✅ **All Classes in Java Inherit From Object Class**

- Object is the **superclass of all classes** in Java.
- Defined in the java.lang package.

🔹 **Common Methods from Object:**

Method	Purpose
toString()	Converts object to string
equals()	Compares two objects logically
hashCode()	Returns hash code of object
getClass()	Returns runtime class info
clone()	Creates and returns copy of object
finalize()	Called by GC before object deletion

```
Student s = new Student();
System.out.println(s.getClass()); // class Student
System.out.println(s.toString()); // Student@hashcode
```

✅ **Java Class Structure Example**

```
package mypack;

public class Student {
  int roll;
  String name;

  void show() {
    System.out.println(roll + " " + name);
  }

  public static void main(String[] args) {
    Student s1 = new Student(); // object creation
    s1.roll = 101;
    s1.name = "Yash";
    s1.show(); // 101 Yash
  }
}
```

✅ **Important Class & Object Points for MCQs**

- Object class is part of **java.lang** package.
- All classes implicitly extend Object.
- Object is created using new keyword.
- getClass() method returns class name at runtime.
- A class can have fields, methods, constructors, blocks, inner classes.

🧠 **Quick MCQ Flash:**

1. What is the parent class of every Java class? → Object
2. Which package contains Object class? → java.lang
3. Can we create object without class? → ❌ No
4. What keyword is used to create an object? → new
5. How many objects can be created from a class? → Unlimited

✅ **Java Constructors – Detailed Notes**

🔹 **What is a Constructor?**

- A **constructor** is a **special method** used to initialize objects.
- It is **called automatically** when an object is created.
- Constructor name = class name.
- **No return type**, not even void.

🔹 **Types of Constructors**

1. Default Constructor

- No parameters.
- Provided by compiler **if no constructor is defined**.

```
class A {
  int x;
  A() {
    x = 10;
  }
}
```

2. Parameterized Constructor

- Takes parameters to initialize values.

```
class A {
  int x;
  A(int val) {
    x = val;
  }
}
```

3. Copy Constructor *(Not built-in like C++)*

- Manually defined to copy values from another object.

```
class A {
  int x;
  A(A obj) {
    this.x = obj.x;
  }
}
```

✅ **Constructor Rules (Very Important – Often Asked in MCQs)**

1. **Constructor name must match class name.**
2. **No return type allowed.** (Even void not allowed)
3. **Can be overloaded** (i.e., multiple constructors in one class with different parameters).
4. **If no constructor is defined, compiler provides a default one.**
   But **if any constructor is defined**, then default is **not provided**.

🔹 **Constructor Overloading**

```
class A {
  A() { System.out.println("Default"); }
  A(int x) { System.out.println("Parameterized: " + x); }
}
```

🧠 **MCQ Tip**: Constructor overloading is possible using different parameter types and counts.

✅ **Garbage Collection in Java – Detailed Notes**

🔹 **What is Garbage Collection?**

- **Automatic memory management** in Java.
- Unused objects (no reference) are **automatically destroyed** by the JVM to **free heap memory**.

🔹 **When is an Object Eligible for GC?**

- When **no live reference** is pointing to it.

```
Student s = new Student();
s = null; // eligible for GC
or
Student s1 = new Student();
Student s2 = new Student();
s1 = s2; // original object referenced by s1 is now unreferenced
```

✅ **How GC Works Internally**

1. **Mark and Sweep Algorithm**
   - **Mark** phase: JVM identifies all referenced objects.
   - **Sweep** phase: Removes all **unreferenced** objects.
2. **GC is non-deterministic**: You can't predict when it will run.

🔹 **System.gc() and Runtime.gc()**

- These are **requests**, not commands. JVM **may or may not** run GC.

```
System.gc();  // Hint to JVM
```

🔹 **finalize() Method (Deprecated from Java 9+)**

- Called **before object is garbage collected**.

```
protected void finalize() throws Throwable {
  System.out.println("Object is being collected");
}
```

🧠 Not reliable for resource cleanup. Use **try-with-resources** or close() methods for cleanup.

✅ **Important Points for MCQs:**

Concept	Detail
GC manages which memory?	**Heap memory**
When is object eligible for GC?	When it's **not referenced**
Which method was used before GC?	finalize() (now deprecated)
Can we force GC?	❌ No, only **request** via System.gc()
When is default constructor given?	Only if **no constructor is defined** by programmer
Constructor return type?	**None** (not even void)
Parent class of all classes?	Object
Memory leak possible in Java?	Rare, but possible if references are **not released**

✅ **Java Methods vs Constructors**
✅ **Method Overloading vs Overriding**
✅ **Rules for Overloading & Overriding**
✅ **Checked Exceptions in Overriding**
✅ **Covariant Return Types**
✅ **Final Summary Table + MCQ Tips**

✅ **1. Java Constructor vs Method**

Feature	Constructor	Method
Purpose	Initializes object	Defines behavior or logic
Name	Must match class name	Any valid identifier
Return type	❌ No return type (not even void)	✅ Must have a return type
Invocation	Automatically called during object creation	Called explicitly via object reference
Can be inherited?	❌ No	✅ Yes
Can be overridden?	❌ No	✅ Yes
Can be overloaded?	✅ Yes	✅ Yes

✅ **2. Method Overloading (Compile-Time Polymorphism)**

🔹 **Definition:**

Same method name with **different parameter list** in the same class.

```
class A {
  void show() {}
  void show(int x) {}
  void show(String s, int x) {}
}
```

🔹 **Rules for Overloading:**

1. Must differ in **type or number** of parameters.
2. Can differ in **return type**, but **not only return type**.

```
void show(int x) {}
int show(int x) {} // ❌ Error: same method signature
```

3. **Static methods** can also be overloaded.

✅ **3. Method Overriding (Run-Time Polymorphism)**

🔹 **Definition:**

Child class provides specific implementation for a method in the parent class.

```
class Parent {
  void show() { System.out.println("Parent"); }
}

class Child extends Parent {
  @Override
  void show() { System.out.println("Child"); }
}
```

🔹 **Rules for Overriding:**

Rule	Description
Method signature	Must be **same** (name + params)
Access modifier	Can't be more restrictive
Return type	Must be same or **covariant** (child type)
Exceptions	Can throw **narrower or same** checked exceptions
Static/final/private	Cannot override **static**, **final**, or **private** methods
Constructor	❌ Constructors **can't be overridden**

```
// Invalid: static method
static void show() {} // hides, not overrides

// Valid override with covariant return
class Animal {}
class Dog extends Animal {}

class A {
  Animal get() { return new Animal(); }
}
class B extends A {
  Dog get() { return new Dog(); } // covariant return
}
```

✅ **4. Checked Exceptions in Overriding**

🔹 **Rule:**

- **Child class overridden method**:
  - ❌ Cannot throw **broader** checked exceptions than parent.

- ✅ Can throw **fewer or narrower** checked exceptions.

```
class A {
  void show() throws IOException {}
}

class B extends A {
  void show() throws FileNotFoundException {} // ✅ narrower
  // void show() throws Exception {}       // ❌ broader
}
```

🔹 **Unchecked Exceptions (like NullPointerException) → No restriction.**

✅ **5. Covariant Return Type**

🔹 **Definition:**

In overriding, **child class can change return type** to a **subclass** of parent method's return type.

```
class Animal {}
class Dog extends Animal {}

class Parent {
  Animal getAnimal() { return new Animal(); }
}

class Child extends Parent {
  @Override
  Dog getAnimal() { return new Dog(); } // ✅ valid covariant return
}
```

🧠 MCQ Tip:

- Only works for **non-primitive** return types.
- Works with **method overriding**, **not overloading**.

✅ **6. Final Summary Table**

Feature	**Overloading**	**Overriding**
Class relationship	Same class	Parent-child (inheritance required)
Method signature	Must be different	Must be exactly same
Return type	Can differ	Same or covariant (child type)
Access modifier	Any	Cannot reduce visibility
Static methods	Can be overloaded	❌ Cannot override, only hidden
Final/private methods	Can be overloaded	❌ Cannot be overridden
Exceptions	Irrelevant	Only same or narrower checked exceptions
Runtime/Compile-time	Compile-time	Runtime

🧠 **MCQ Flash Tips:**

1. Method overloading → **compile-time polymorphism**
2. Method overriding → **runtime polymorphism**
3. Covariant return types only apply to **non-primitives**
4. final, static, private methods → **not overridden**
5. Overriding can't throw **broader checked exceptions**
6. Constructor can't be **inherited or overridden**

✅ **static Keyword in Java – Full Notes**

🔹 **What is static?**

- static is a **modifier** used for **memory-efficient programming**.
- It means the member **belongs to the class**, not to any specific object.
- Common to all instances (shared).

✅ **Where can you use static?**

Usage	Example
Static variable	static int count = 0;
Static method	static void display()
Static block	static { ... }
Static class	static class InnerClass {}

✅ **1. Static Variables (a.k.a Class Variables)**

- Belongs to the **class**, not object.
- **Only one copy** shared by all objects.
- Useful for **common counters, constants, etc.**

```
class Counter {
  static int count = 0;
  Counter() { count++; }
}
Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.count); // Output: 2
```

✅ **2. Static Methods**

- Can be called **without creating object**.
- Can access only **static variables and other static methods**.
- Cannot use this or super.

```
class MathUtil {
  static int square(int x) {
    return x * x;
  }

  public static void main(String[] args) {
    System.out.println(MathUtil.square(5)); // 25
  }
```

```
}
```

✅ **3. Static Block**

- Runs **once** when the class is loaded.
- Used for **static initialization**.

```
class Test {
  static int x;
  static {
    x = 10;
    System.out.println("Static Block");
  }

  public static void main(String[] args) {
    System.out.println(Test.x); // 10
  }
}
```

🧠 Runs **before main()** if class is loaded first.

✅ **4. Static Nested Class**

- A **nested class** declared static.
- Can access only **static members** of outer class.

```
class Outer {
  static int data = 30;

  static class Inner {
    void msg() {
      System.out.println("data is " + data);
    }
  }

  public static void main(String args[]) {
    Outer.Inner obj = new Outer.Inner();
    obj.msg(); // data is 30
  }
}
```

✅ **5. Restrictions with static**

- static methods **cannot access non-static members** directly.
- **Cannot use this or super** inside static methods.
- Instance methods can access **both static and non-static** members.

✅ **Important MCQ Tips:**

Statement	Valid?
Static methods can be overloaded	✅ Yes
Static methods can be overridden	❌ No (they are hidden)
Static block can access static variables	✅ Yes
You can use this in a static method	❌ No
Static nested class can access non-static data	❌ No
Static variables are stored in method stack	❌ No, stored in Method Area

---

🧠 **Real-World Uses:**

- Utility methods (e.g., Collections.sort())
- Constants (public static final)
- Singleton pattern (private static instance)
- Static blocks for loading drivers, configurations

✅ **1. this Keyword in Java**

🔹 **Purpose:**

Refers to the **current object** of the class.

🔹 **Common Uses of this:**

Use Case	Example
Refer current class instance variable	this.x = x;
Invoke current class constructor	this()
Pass current object as argument	someMethod(this);
Return current object	return this;
Call current class method explicitly	this.methodName();

🔸 **Example: this for variable differentiation**

```
class Student {
  int id;
  Student(int id) {
    this.id = id; // differentiates instance variable from parameter
  }
}
```

🔸 **Example: Constructor chaining using this()**

```
class A {
  A() {
    this(5); // calls parameterized constructor
    System.out.println("Default Constructor");
  }
  A(int x) {
    System.out.println("Parameterized Constructor: " + x);
  }
}
```

🔹 **MCQ Tip:**

- this() must be the **first statement** in constructor.
- You can't use this() and super() together.

✅ 2. final Keyword in Java

🔹 **Purpose:**

Used to declare **constants**, **prevent inheritance**, or **prevent method overriding**.

🔹 **final Can Be Applied To:**

Context	Effect
final variable	Value **cannot be changed** after initialization
final method	**Cannot be overridden** in subclass
final class	**Cannot be extended** (inherited)

🔸 **Example: Final variable**

```
final int MAX = 100;
// MAX = 200; // ❌ Error
```

🔸 **Example: Final method**

```
class A {
  final void show() {}
}
class B extends A {
  // void show() {} ❌ Error: cannot override final method
}
```

🔸 **Example: Final class**

```
final class A {}
// class B extends A {} ❌ Error: cannot inherit final class
```

🔹 **MCQ Tip:**

- final is used for **immutability** and **restriction**.
- Local final variables must be initialized before use.
- Final references can't be reassigned, but the object's content **can change**.

✅ 3. super Keyword in Java

🔹 **Purpose:**

Used to **access parent class** members.

🔹 **Common Uses of super:**

Use Case	Example
Access parent class variable	super.x
Call parent class method	super.method()
Invoke parent class constructor	super()

🔸 **Example: Accessing parent variable**

```
class A {
  int x = 10;
}
```

```
class B extends A {
  int x = 20;
  void show() {
    System.out.println(super.x); // prints 10
  }
}
```

🔸 **Example: Calling parent constructor**

```
class A {
  A() {
    System.out.println("Parent Constructor");
  }
}
class B extends A {
  B() {
    super(); // calls parent constructor
    System.out.println("Child Constructor");
  }
}
```

🔹 **MCQ Tip:**

- super() must be the **first statement** in child constructor.
- You **can't use super() and this() together** in one constructor.

✅ **Summary Table**

Keyword	Purpose	Usable In	Restrictions
this	Refers to current object	Instance methods/constructors	this() must be 1st in constructor
final	Make things unchangeable	Variables, Methods, Class	Cannot reassign/override/extend
super	Refers to immediate parent class	Subclass constructor/method	super() must be 1st in constructor

---

✅ **Bonus MCQs:**

1. Can we override a final method? → ❌ No
2. Can a final class be extended? → ❌ No
3. Can a final variable be assigned twice? → ❌ No
4. Can this() and super() be used together? → ❌ No
5. Does super() call default parent constructor? → ✅ Yes

In Java, this() and super() **cannot be used together in the same constructor**, because **both must be the first statement**, and **a constructor can only have one first statement**.

🚫 **Invalid Java Code (Compiler Error): this() + super() together**

```
class Parent {
  Parent() {
    System.out.println("Parent constructor");
  }
}

class Child extends Parent {
  Child() {
    this(10);     // ❌ this() is the first statement
    super();      // ❌ Error: can't have super() after this()
    System.out.println("Child constructor");
  }

  Child(int x) {
    System.out.println("Child constructor with int: " + x);
  }

  public static void main(String[] args) {
    new Child();
  }
}
```

✅ **Compiler Error:**

Constructor call must be the first statement in a constructor

---

✅ **Correct Usage Example (Choose One Only):**

✓ **Using super() only:**

```
class Parent {
  Parent() {
    System.out.println("Parent constructor");
  }
}

class Child extends Parent {
  Child() {
    super(); // OK
    System.out.println("Child constructor");
  }
}
```

✓ **Using this() only:**

```
class Child {
```

```
  Child() {
    this(10); // OK
    System.out.println("Default constructor");
  }

  Child(int x) {
    System.out.println("Parameterized constructor: " + x);
  }
}
```

🧠 **Key MCQ Rule:**

- this() → Calls another constructor in **same class**
- super() → Calls constructor of **parent class**
- ❌ Only one can be the **first line** in a constructor.

✅ **1. Java Variables and Literals**

🔹 **Variables:**

- A **variable** is a name given to a memory location that holds a value.

🔸 **Types of Variables:**

Type	Scope & Use
Local	Declared inside methods or blocks
Instance	Non-static; belongs to object
Static/Class	Shared by all objects (use static)

```
int a = 10; // local variable
```

🔹 **Literals:**

Fixed values assigned to variables.

Type	Example
Integer	10, 0b1010, 0xFF
Floating Point	10.5, 3.14f
Boolean	true, false
Character	'A', '\n'
String	"Hello"

✅ **2. Java Data Types (Primitive)**

Java has 8 primitive data types:

Type	Size	Default	Example
byte	1 byte	0	byte b = 100;
short	2 bytes	0	short s = 20;
int	4 bytes	0	int x = 1000;
long	8 bytes	0L	long l = 123L;
float	4 bytes	0.0f	float f = 10.5f;
double	8 bytes	0.0	double d = 12.3;

Type	Size	Default	Example
char	2 bytes	\u0000	char c = 'A';
boolean	1 bit	false	boolean b = true;

🧠 **Note**: Floating point literals default to double. Use f or F for float.

✅ **3. Java Operators**

🔹 **Types of Operators:**

Category	Operators
Arithmetic	+, -, *, /, %
Relational	==, !=, >, <, >=, <=
Logical	&&, `
Assignment	=, +=, -=, *=, /=, %=
Unary	+, -, ++, --, !
Bitwise	&, `
Ternary	? : (conditional)
Instanceof	Checks object type (obj instanceof Class)

🔸 **Example:**

```
int a = 5, b = 3;
System.out.println(a + b);     // 8
System.out.println(a > b);     // true
System.out.println(a & b);     // 1 (binary AND)
```

✅ **4. Java Basic Input and Output**

🔹 **Input using Scanner:**

```
import java.util.Scanner;

public class Main {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int x = sc.nextInt();
    String name = sc.next();
  }
}
```

🔹 **Output using System.out:**

```
System.out.print("Hello ");     // no newline
System.out.println("World");    // with newline
```

✅ **5. Java Expressions, Statements, and Blocks**

🔹 **Expressions:**

A combination of variables, operators, and values that evaluates to a result.

```
int result = (3 + 4) * 2;  // Expression
```

🔹 **Statements:**

An instruction to the compiler to perform an action.

```
int x = 10;      // Declaration statement
x++;             // Expression statement
System.out.println(x);  // Method call statement
```

🔹 **Blocks:**

A group of zero or more statements enclosed in {}. Used in if, loops, methods, etc.

```
{
  int a = 10;
  System.out.println(a);
}
```

🧠 **MCQ Tips Summary:**

1. Java has **8 primitive data types**.
2. float literals must end with f or F.
3. System.out.println() is used for output.
4. Scanner is commonly used for input.
5. % is the **modulus operator** (remainder).
6. Expressions evaluate to values; statements perform actions.

✅ **1. if...else Statement**

🔹 **Syntax:**

```
if (condition) {
  // if block
} else {
  // else block
}
```

🔹 **Example:**

```
int age = 20;
if (age >= 18) {
  System.out.println("Adult");
} else {
  System.out.println("Minor");
}
```

✅ **MCQ Tips:**

- Curly braces {} are optional for single statements.
- if-else if-else creates a decision chain.
- Conditions must return boolean.

✅ **2. Ternary Operator (? :)**

🔹 **Syntax:**

```
variable = (condition) ? value_if_true : value_if_false;
```

🔹 **Example:**

```
int age = 20;
String result = (age >= 18) ? "Adult" : "Minor";
System.out.println(result);
```

✅ **MCQ Tips:**

- It's a **shortcut to if...else**.

- It **returns a value**, so can be assigned.

✅ **3. for Loop**

🔹 **Syntax:**

```
for (initialization; condition; update) {
  // body
}
```

🔹 **Example:**

```
for (int i = 1; i <= 5; i++) {
  System.out.println(i);
}
```

✅ **MCQ Tips:**

- Executes **fixed number of times**.
- All three parts are optional: for(;;) = infinite loop.
- You can declare loop variable inside or outside.

✅ **4. for-each Loop (Enhanced For Loop)**

🔹 **Syntax:**

```
for (data_type var : array_or_collection) {
  // body
}
```

🔹 **Example:**

```
int[] arr = {10, 20, 30};
for (int val : arr) {
  System.out.println(val);
}
```

✅ **MCQ Tips:**

- Cannot modify array directly.
- Best for reading, not for updating index-based values.

✅ **5. while and do...while Loops**

🔹 **while Loop:**

```
while (condition) {
  // body
}
```

🔹 Example:

```
int i = 1;
while (i <= 3) {
  System.out.println(i);
  i++;
}
```

🔹 **do...while Loop:**

```
do {
  // body
} while (condition);
```

🔹 Example:

```
int i = 1;
do {
  System.out.println(i);
  i++;
} while (i <= 3);
```

✅ **MCQ Tips:**

Loop Type	Condition Check	Executes at least once?
while	Before loop body	❌ No
do...while	After loop body	✅ Yes

✅ 6. break Statement

🔹 **Use:**

Exits the loop or switch block immediately.

🔹 **Example:**

```
for (int i = 1; i <= 5; i++) {
  if (i == 3) break;
  System.out.println(i);
}
// Output: 1 2
```

✅ **MCQ Tips:**

- Often used with switch, for, while, do-while.
- Exits **only the nearest loop or switch**.

✅ 7. continue Statement

🔹 **Use:**

Skips **current iteration** and continues with next.

🔹 **Example:**

```
for (int i = 1; i <= 5; i++) {
  if (i == 3) continue;
  System.out.println(i);
}
// Output: 1 2 4 5
```

✅ **MCQ Tips:**

- continue skips **to next iteration** of the loop.
- In for loop, the **increment part still executes** after continue.

✅ 8. switch Statement

🔹 **Syntax:**

```
switch (expression) {
  case value1:
    // block
    break;
  case value2:
    // block
    break;
```

```
  default:
    // block
}
```

🔹 **Example:**

```
int day = 2;
switch (day) {
  case 1: System.out.println("Sun"); break;
  case 2: System.out.println("Mon"); break;
  default: System.out.println("Invalid");
}
```

✅ **MCQ Tips:**

- Works with: int, byte, short, char, String, and enum (Java 7+).
- Each case must be a **constant** or final variable.
- break prevents **fall-through**.
- default is optional but recommended.

📌 **Summary Chart for MCQs:**

Statement Type	Key Feature
if...else	Works on boolean condition
?: (ternary)	Short form of if...else, returns value
for	Use when iterations are **fixed**
while	Condition checked **before loop starts**
do...while	Executes **at least once**
for-each	Best for arrays/collections (read-only)
break	Exits loop or switch
continue	Skips to next iteration
switch	Best for **multi-way** decision based on value

✅ Java Arrays
✅ Multidimensional Arrays
✅ Array Copying
✅ With Code Examples + MCQ Tips

✅ **1. Java Arrays**

🔹 **What is an Array?**

- A **container object** that holds **fixed-size elements of the same type**.
- Stored in **contiguous memory**.
- Can hold **primitive or reference** types.

🔹 **Declaration & Initialization**

```
int[] arr1;        // Declaration
arr1 = new int[5];   // Memory allocation

int[] arr2 = {10, 20, 30, 40};  // Declaration + Initialization
```

✅ Default values:

Type	Default
int	0
float	0.0f
boolean	false
Object	null

🔹 **Accessing Array Elements**

```
System.out.println(arr2[2]);  // Output: 30
```

🔸 Looping through array:

```
for (int i = 0; i < arr2.length; i++) {
   System.out.println(arr2[i]);
}

// Enhanced for loop
for (int value : arr2) {
   System.out.println(value);
}
```

✅ **MCQ Tips – Arrays (1D):**

- Index starts from **0**, last index is length - 1
- Array size is **fixed after declaration**
- Arrays are **objects** in Java
- arr.length is a **property**, not a method (arr.length() ❌)
- Accessing out-of-bound index throws ArrayIndexOutOfBoundsException

---

✅ **2. Multidimensional Arrays (2D, 3D...)**

🔹 **Declaration:**

```
int[][] matrix = new int[3][3];    // 3x3 matrix
int[][] jagged = new int[3][];     // jagged array
```

🔹 **Initialization:**

```
int[][] mat = {
   {1, 2, 3},
   {4, 5, 6}
};
```

🔹 **Accessing Elements:**

```
System.out.println(mat[1][2]);  // Output: 6
```

🔹 **Nested Loops:**

```
for (int i = 0; i < mat.length; i++) {
   for (int j = 0; j < mat[i].length; j++) {
      System.out.print(mat[i][j] + " ");
   }
   System.out.println();
}
```

✅ **MCQ Tips – Multidimensional Arrays:**

- 2D arrays are arrays **of arrays**
- Jagged arrays can have **unequal row lengths**
- int[][] arr = new int[2][]; is valid
- Default value for elements is based on type (e.g., 0 for int)

✅ **3. Copying Arrays in Java**

🔹 **1. Manual Copy (loop)**

```
int[] src = {1, 2, 3};
int[] dest = new int[src.length];

for (int i = 0; i < src.length; i++) {
   dest[i] = src[i];
}
```

🔹 **2. Using System.arraycopy()**

```
System.arraycopy(src, 0, dest, 0, src.length);
```

Param	Meaning
src	source array
0	source starting index
dest	destination array
0	destination starting index
src.length	number of elements to copy

🔹 **3. Using Arrays.copyOf() (Java 6+)**

```
import java.util.Arrays;
int[] copied = Arrays.copyOf(src, src.length);
```

🔹 **4. Using clone()**

```
int[] cloned = src.clone();
```

🔸 Deep copy for primitives

🔸 For object arrays, it's **shallow copy** (references copied)

✅ **MCQ Tips – Copying Arrays:**

- System.arraycopy() is **fastest**
- clone() creates a **shallow copy**
- Arrays.copyOf() creates a new array with specified length
- Changing src[] doesn't affect dest[] (if primitive)

✅ **Quick Summary Table**

Feature	Key Points
Array	Fixed-size, homogeneous elements
arr.length	Returns size (no parentheses)
2D Array	Array of arrays; can be jagged

Feature	Key Points
Manual Copy	Using for loop
System.arraycopy()	Fastest built-in method
Arrays.copyOf()	Returns new array, allows resizing
clone()	Clones array; shallow copy for objects
Default Values	0, false, null, etc., depending on data type

✅ **Sample MCQs:**

Q1. What is the default value of elements in an int array?

✅ 0

Q2. Which method performs a shallow copy of an object array?

✅ .clone()

Q3. Which of the following can resize an array?

✅ Arrays.copyOf()

Q4. What does arr.length return?

✅ Number of elements in array

Q5. What exception is thrown on invalid index?

✅ ArrayIndexOutOfBoundsException

✅ **1. Java Inheritance**

🔹 **What is it?**

Inheritance allows one class (child) to **acquire properties and behaviors** (methods & fields) from another class (parent).

🔹 **Syntax:**

```
class Parent {
  int x = 10;
}
class Child extends Parent {
  int y = 20;
}
```

🔹 **Types in Java:**

Type	Support in Java
Single Inheritance	✅ Yes
Multilevel	✅ Yes
Hierarchical	✅ Yes
Multiple	❌ No (but via Interface)

✅ **MCQ Tips:**

- Inheritance is implemented using extends
- Constructors are **not inherited**
- Java supports **single class inheritance** (not multiple)
- Object is the **root class** of all Java classes

✅ **2. Java Method Overriding**

🔹 **What is it?**

Redefining a method of parent class in a child class with **same signature**.

🔹 **Rules:**

- Same method name, return type, and parameters
- Must be in **subclass**
- Cannot override:
  - final methods
  - static methods
  - constructors
  - private methods

🔹 **Example:**

```
class Parent {
  void show() {
    System.out.println("Parent");
  }
}
class Child extends Parent {
  @Override
  void show() {
    System.out.println("Child");
  }
}
```

✅ **MCQ Tips:**

- Use @Override annotation
- Return type must be same or **covariant**
- Overriding is **runtime polymorphism**
- Access modifier cannot be **more restrictive**

✅ **3. Java super Keyword**

🔹 **Uses:**

Purpose	Example
Access parent field	super.x
Call parent method	super.method()
Call parent constructor	super()

🔹 **Rules:**

- super() must be **first line** in constructor
- Cannot be used in **static context**

```
class Parent {
  int x = 10;
}
class Child extends Parent {
  int x = 20;
  void display() {
    System.out.println(super.x);  // prints 10
  }
```

}

✅ **4. Abstract Class and Methods**

🔹 **Abstract Class:**

- Declared using abstract keyword
- Cannot be instantiated
- May have **abstract and non-abstract** methods

🔹 **Abstract Method:**

- No body; must be overridden
- Declared with abstract keyword

🔹 **Example:**

```
abstract class Animal {
  abstract void sound();
  void eat() {
    System.out.println("Eating");
  }
}
class Dog extends Animal {
  void sound() {
    System.out.println("Bark");
  }
}
```

✅ **MCQ Tips:**

- An abstract class **can** have constructors
- Cannot create object of abstract class
- Abstract method **must** be in abstract class
- Class becomes abstract if it has **even one** abstract method

---

✅ **5. Java Interface**

🔹 **Interface:**

- Blueprint of class with only **abstract methods (Java 7)** or **default/static methods (Java 8+)**
- All fields are public static final by default
- Methods are public abstract (unless default/static)

🔹 **Syntax:**

```
interface Flyable {
  void fly();
}
class Bird implements Flyable {
  public void fly() {
    System.out.println("Flying");
  }
}
```

✅ **Java 8+ Features:**

- default methods with body
- static methods in interface

✅ **MCQ Tips:**

- Interface cannot have constructors
- A class can implement **multiple interfaces**
- Interface supports **multiple inheritance**
- Interface methods are implicitly public abstract
- Interface variables are public static final

✅ 6. Java Polymorphism

🔹 **What is it?**

"Many forms" – ability of object to take multiple forms

🔹 **Types:**

Type	Description
Compile-time	Method Overloading
Runtime	Method Overriding

🔹 **Example (Runtime):**

```
class A {
  void show() { System.out.println("A"); }
}
class B extends A {
  void show() { System.out.println("B"); }
}

A obj = new B();
obj.show();  // Output: B
```

✅ **MCQ Tips:**

- Overloading = compile-time polymorphism
- Overriding = runtime polymorphism
- Runtime polymorphism uses **method overriding and upcasting**
- Constructors are **not polymorphic**

✅ 7. Java Encapsulation

🔹 **What is it?**

Hiding internal details and exposing only necessary information via **getters and setters**.

🔹 **Achieved by:**

- Declaring variables private
- Providing public getter and setter methods

```
class Student {
  private int age;

  public int getAge() {
    return age;
  }

  public void setAge(int age) {
    this.age = age;
```

```
  }
}
```

✅ **Benefits:**

- Data hiding
- Code maintainability
- Improves security
- Loose coupling

✅ **MCQ Tips:**

- Encapsulation = binding data + methods
- Achieved using private + getter/setter
- Improves code modularity

🧠 **Summary Table**

Concept	Key Notes
Inheritance	extends keyword; reuse code
Overriding	Same signature, subclass
super	Refers to parent
Abstract	abstract keyword; must override
Interface	implements; full abstraction
Polymorphism	Many forms (overloading, overriding)
Encapsulation	private data + public methods

✅ **Sample MCQs:**

1. Can we create an object of abstract class? → ❌ No
2. What is used to inherit a class? → ✅ extends
3. Which type of polymorphism is achieved by method overloading? → ✅ Compile-time
4. Can interfaces have constructors? → ❌ No
5. super() must be the: → ✅ First statement in constructor
6. Which class is the parent of all Java classes? → ✅ Object
7. Can abstract class have constructor? → ✅ Yes
8. Variables in interface are by default: → ✅ public static final

---

✅ Abstract Class vs Interface in Java

Feature	Abstract Class	Interface
Keyword	abstract	interface
Inheritance	Use extends (only one allowed)	Use implements (multiple allowed)
Method Types	Can have both **abstract** and **concrete** methods	Only **abstract** methods (Java 7) Can also have default and static methods (Java 8+)
Variables	Can have instance variables	All variables are public static final
Constructors	✅ Yes	❌ No
Multiple Inheritance	❌ No	✅ Yes (via multiple interfaces)
Access Modifiers	Can use private, protected, etc.	All methods are **public** by default
Usage	Partial abstraction	Full abstraction
Performance	Slightly faster	Slightly slower due to indirection
When to Use	Common behavior with base implementation	Unrelated classes with common contract

---

✅ Example Comparison:

🔹 Abstract Class Example

```
abstract class Animal {
  abstract void sound();  // abstract method
  void eat() {
    System.out.println("Eating");
  }
}
class Dog extends Animal {
  void sound() {
    System.out.println("Bark");
  }
}
```

🔹 Interface Example

```
interface Flyable {
  void fly(); // implicitly public and abstract
}

class Bird implements Flyable {
  public void fly() {
    System.out.println("Flying high");
  }
}
```

🧠 MCQ Tips:

1. **Q:** Can we create an object of an abstract class?
   ✅ No
2. **Q:** Can a class implement multiple interfaces?
   ✅ Yes
3. **Q:** Can interfaces have static methods?
   ✅ Yes (Java 8+)
4. **Q:** Can abstract classes have constructors?
   ✅ Yes
5. **Q:** Interface variables are by default?
   ✅ public static final
6. **Q:** Can we have private methods in interface?
   ✅ Yes (Java 9+), but only for internal use.
7. **Q:** Can we extend multiple abstract classes?
   ❌ No (only one class can be extended)

✅ **Summary Chart**

Use Abstract Class When	Use Interface When
You want to provide default behavior	You want to define a contract without implementation
You need to share code across several classes	You want multiple inheritance of type
You want to define non-static, non-final fields	You need constants only

---

✅ **1. Binding in Java (Static vs Dynamic)**
✅ **2. Lazy vs Eager Initialization**
✅ **3. Core OOPs Concepts in Detail**
✅ **MCQ-Friendly Tips**

✅ **1. Binding in Java**

Binding means linking a method call to its method definition.

🔹 **Types of Binding:**

Type	Also Called	When it Happens	Example
Static Binding	Early Binding	Compile time	Method Overloading
Dynamic Binding	Late Binding	Runtime	Method Overriding (Polymorphism)

🔸 **Static Binding Example:**

```
class Demo {
  void show(int a) {
    System.out.println("int");
  }
  void show(double a) {
    System.out.println("double");
  }
```

```
}

Demo d = new Demo();
d.show(5);  // Compiler decides which method (int)
```

🔸 **Dynamic Binding Example:**

```
class A {
  void display() {
    System.out.println("A");
  }
}
class B extends A {
  void display() {
    System.out.println("B");
  }
}

A obj = new B();  // Upcasting
obj.display();    // Output: B (runtime binding)
```

✅ **MCQ Tips for Binding:**

- **Static binding** → method overloading, final/private/static methods
- **Dynamic binding** → method overriding using inheritance + polymorphism
- Constructors are always **statically bound**
- If method is **private/final/static**, it uses **static binding**

✅ **2. Lazy vs Eager Initialization**

These terms are used in Singleton pattern, object creation, caching, or ORM like Hibernate.

🔹 **Eager Initialization:**

- Object is created **at the time of class loading**
- Simple but not memory efficient

```
class Singleton {
  private static Singleton instance = new Singleton(); // created eagerly
  private Singleton() {}
  public static Singleton getInstance() {
    return instance;
  }
}
```

🔹 **Lazy Initialization:**

- Object is created **when needed**
- Efficient in terms of memory

```
class Singleton {
  private static Singleton instance;
  private Singleton() {}
  public static Singleton getInstance() {
    if (instance == null)
      instance = new Singleton();
```

```
        return instance;
    }
}
```

✅ **MCQ Tips for Lazy vs Eager:**

- Lazy loading saves **resources**, but needs synchronization for **thread safety**
- Eager loading uses **memory even if not needed**
- Hibernate default fetching is **lazy**
- Lazy → **on demand**, Eager → **immediate**

✅ **3. OOPs Concepts in Java**

Java follows **Object-Oriented Programming System (OOPS)**, which includes:

🔹 **1. Encapsulation**

- Wrapping variables + methods in a class
- Achieved using private + public getters/setters

```
class Student {
  private int age;
  public void setAge(int age) { this.age = age; }
  public int getAge() { return age; }
}
```

🧠 **MCQ Tip**: Encapsulation = data hiding

🔹 **2. Inheritance**

- extends keyword
- Code reuse, IS-A relationship

```
class Animal {}
class Dog extends Animal {}
```

🧠 **MCQ Tip**: Java supports **single inheritance**, but **multiple interfaces**

🔹 **3. Polymorphism**

- "Many forms"
- **Compile-time**: Method Overloading
- **Runtime**: Method Overriding

```
// Overriding (runtime)
class A {
  void show() {}
}
class B extends A {
  void show() {}
}
```

🧠 **MCQ Tip**: Dynamic binding = runtime polymorphism

🔹 **4. Abstraction**

- Hiding internal details
- Achieved by **abstract class** or **interface**

```
abstract class Shape {
  abstract void draw();
}
```

🧠 **MCQ Tip**: Interface = full abstraction; Abstract class = partial abstraction

---

🔹 **5. Association, Aggregation, Composition (Advanced)**

Relationship Type		Lifespan
Association	"uses-a"	Loose
Aggregation	"has-a"	Independent
Composition	"owns-a"	Dependent

✅ **Final Summary Table**

Concept	Keyword / Use	Key Notes
Binding	static / dynamic	Overloading = static; Overriding = dynamic
Lazy Loading	if(instance == null)	Efficient, used on demand
Encapsulation	private, get/set	Data hiding
Inheritance	extends	Reuse code
Polymorphism	overload/override	Many forms
Abstraction	abstract/interface	Hides details

✅ **Sample MCQs:**

1. What type of binding is used for method overriding?
   ✅ Dynamic Binding
2. Which type of polymorphism uses upcasting and late binding?
   ✅ Runtime Polymorphism
3. Singleton with delayed object creation uses?
   ✅ Lazy Initialization
4. Which OOP concept hides internal details?
   ✅ Abstraction
5. Inheritance is achieved using:
   ✅ extends
6. Which is used for compile-time polymorphism?
   ✅ Method Overloading
7. Private methods use which binding?
   ✅ Static Binding

✅ **Upcasting**
✅ **Downcasting**
✅ **instanceof Operator**
✅ **MCQ Tips + Code Examples**

✅ 1. Upcasting in Java

🔹 **Definition:**

Converting a subclass type to its superclass type.

✅ **Safe and implicit** (automatic)

🔹 **Syntax:**

```
Parent obj = new Child();  // ✅ Upcasting
```

🔹 **Example:**

```
class Animal {
  void sound() {
    System.out.println("Animal sound");
  }
}
class Dog extends Animal {
  void bark() {
    System.out.println("Dog barks");
  }
}
public class Test {
  public static void main(String[] args) {
    Animal a = new Dog();  // ✅ Upcasting
    a.sound();          // Allowed
    // a.bark(); ❌ Not allowed
  }
}
```

🔹 **Key Points:**

- Only superclass methods are accessible
- Object remains of Child type at runtime
- Used for **runtime polymorphism**

✅ 2. Downcasting in Java

🔹 **Definition:**

Converting a superclass reference back to a subclass type.

❌ Needs **explicit cast**

❌ Risky → May throw ClassCastException

🔹 **Syntax:**

```
Child c = (Child) parentRef;  // ✅ Downcasting
```

🔹 **Example:**

```
Animal a = new Dog();      // Upcasting
Dog d = (Dog) a;         // ✅ Downcasting
```

```
d.bark();          // Now works
```

❌ **Incorrect Downcasting:**

```
Animal a = new Animal();
Dog d = (Dog) a;  // ❌ Runtime Error: ClassCastException
```

✅ **3. instanceof Operator**

🔹 **Use:**

Check **object type at runtime** before downcasting.

🔹 **Syntax:**

```
if (obj instanceof ClassName) { ... }
```

🔹 **Example:**

```
Animal a = new Dog();

if (a instanceof Dog) {
  Dog d = (Dog) a;  // Safe downcasting
  d.bark();
}
```

✅ **Output:**

```
Dog barks
```

✅ **When to Use instanceof?**

- To **prevent ClassCastException** during downcasting.
- To perform **type-specific operations**.

✅ **Summary Table**

Concept	Upcasting	Downcasting
Type	Child → Parent	Parent → Child
Syntax	Parent p = new Child();	Child c = (Child) p;
Safety	Safe	Risky (use instanceof)
Access	Parent methods only	Child methods after cast
Binding	Used in dynamic polymorphism	Only useful if object is actual child

---

✅ **MCQ Tips:**

🧠 **Upcasting**

1. **Is upcasting implicit or explicit?**
   ✅ Implicit
2. **Can upcasting access child methods?**
   ❌ No, only superclass methods
3. **What type of polymorphism uses upcasting?**
   ✅ Runtime (Dynamic) Polymorphism

🧠 **Downcasting**

4. **Is downcasting always safe?**
   ❌ No, may cause ClassCastException

5. **How to ensure safe downcasting?**
   ✅ Use instanceof
6. **What happens if you downcast an object wrongly?**
   ✅ ClassCastException at runtime

🧠 **instanceof**

7. **instanceof returns which type?**
   ✅ boolean
8. **Can instanceof be used with null?**
   ✅ Yes; null instanceof AnyClass → ❌ false
9. **Does instanceof check compile-time or runtime type?**
   ✅ Runtime

✅ **Final Recap Code**

```
class Animal {}
class Dog extends Animal {
  void bark() {
    System.out.println("Bark");
  }
}

public class Main {
  public static void main(String[] args) {
    Animal a = new Dog();       // Upcasting
    if (a instanceof Dog) {     // Safe check
      Dog d = (Dog) a;        // Downcasting
      d.bark();
    }
  }
}
```

✅ **1. String in Java**
✅ **2. StringBuilder vs StringBuffer**
✅ **3. String Pool & Heap Memory**
✅ **4. Important Methods (intern(), equals(), ==, etc.)**
✅ **5. MCQ Tips + Code Snippets**

✅ **1. String Class in Java**

🔹 **Overview:**

- String is an **immutable** class in java.lang package.
- Stored in **String Constant Pool (SCP)**.
- Changing a String actually creates a **new object**.

```
String s1 = "hello";
s1 = s1.concat(" world");  // creates new string
System.out.println(s1);    // Output: hello world
```

🔹 **Why String is Immutable?**

- For security (passwords, file paths)
- Thread safety
- Hashcode consistency (used in keys of HashMap)
- SCP reuse

🔹 **Common String Methods:**

Method	Use Example	Output
length()	"abc".length()	3
charAt(int)	"abc".charAt(1)	'b'
substring(int)	"hello".substring(2)	"llo"
toUpperCase()	"hi".toUpperCase()	"HI"
equals()	"abc".equals("abc")	true
equalsIgnoreCase()	"Abc".equalsIgnoreCase("abc")	true
==	Compares references (not content)	false or true
compareTo()	Lexicographic compare	0, +ve, -ve
trim()	Removes leading/trailing spaces	" a ".trim() → "a"
replace()	"java".replace("a", "x")	"jxvx"
contains()	"hello".contains("ll")	true

✅ **2. StringBuilder vs StringBuffer**

Feature	StringBuilder	StringBuffer
Mutability	Mutable	Mutable
Thread Safe	❌ Not thread-safe	✅ Thread-safe (synchronized)
Performance	✅ Faster	❌ Slower
Introduced In	Java 5	Java 1.0
Package	java.lang	java.lang

🔸 **Example – StringBuilder:**

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" Java");
System.out.println(sb);  // Output: Hello Java
```

🔸 **Example – StringBuffer:**

```
StringBuffer sb = new StringBuffer("Thread");
sb.append(" Safe");
System.out.println(sb);  // Output: Thread Safe
```

✅ **3. String Pool vs Heap Memory**

🔹 **String Pool (SCP):**

- Part of **method area / metaspace**
- Holds **string literals** only
- Ensures **memory optimization**

String s1 = "java";
String s2 = "java";
System.out.println(s1 == s2); // ✅ true (same reference)

🔹 **Heap:**

- Stores objects created using new

String s1 = new String("java");
String s2 = new String("java");
System.out.println(s1 == s2); // ❌ false (new objects)

✅ **4. intern() Method**

🔹 **Use:**

Puts string into SCP and returns its reference.
String s1 = new String("core").intern();
String s2 = "core";

System.out.println(s1 == s2); // ✅ true

🔹 **Purpose:**

- Save memory
- Useful in large apps where string duplication is common

✅ **5. Summary Table**

Class	Mutable	Thread Safe	Stored In	Performance
String	❌	✅ (Immutable)	SCP/Heap	Medium (new object each time)
StringBuilder	✅	❌	Heap	✅ Fast
StringBuffer	✅	✅	Heap	Slower

✅ **6. MCQ Tips**

🔹 **String**

1. "abc" == "abc" → ✅ true (both in SCP)
2. new String("abc") == "abc" → ❌ false
3. String is immutable → ✅ true
4. "abc".charAt(3) → ❌ StringIndexOutOfBoundsException

🔹 **StringBuilder / Buffer**

5. Which is faster? → ✅ StringBuilder
6. Which is thread-safe? → ✅ StringBuffer
7. Can StringBuilder be used in multithreaded env? → ❌ No
8. append() is used in → ✅ StringBuilder and StringBuffer

🔹 **String Pool / intern()**

9. What does intern() do?
   ✅ Moves object to String pool if not present
10. "abc".intern() == "abc" → ✅ true

✅ **Practice Code Snippet**

```
public class Test {
  public static void main(String[] args) {
    String s1 = "java";
    String s2 = new String("java");
    String s3 = s2.intern();

    System.out.println(s1 == s2);  // false
    System.out.println(s1 == s3);  // true

    StringBuilder sb = new StringBuilder("Hello");
    sb.append(" World");
    System.out.println(sb);  // Hello World
  }
}
```

✅ **1. String in Java**
✅ **2. StringBuilder vs StringBuffer**
✅ **3. String Pool & Heap Memory**
✅ **4. Important Methods (intern(), equals(), ==, etc.)**
✅ **5. MCQ Tips + Code Snippets**

✅ **1. String Class in Java**

🔹 **Overview:**

- String is an **immutable** class in java.lang package.
- Stored in **String Constant Pool (SCP)**.
- Changing a String actually creates a **new object**.

```
String s1 = "hello";
s1 = s1.concat(" world");  // creates new string
System.out.println(s1);    // Output: hello world
```

🔹 **Why String is Immutable?**

- For security (passwords, file paths)
- Thread safety
- Hashcode consistency (used in keys of HashMap)
- SCP reuse

🔹 **Common String Methods:**

Method	Use Example	Output
length()	"abc".length()	3
charAt(int)	"abc".charAt(1)	'b'
substring(int)	"hello".substring(2)	"llo"
toUpperCase()	"hi".toUpperCase()	"HI"
equals()	"abc".equals("abc")	true
equalsIgnoreCase()	"Abc".equalsIgnoreCase("abc")	true

Method	Use Example	Output
==	Compares references (not content)	false or true
compareTo()	Lexicographic compare	0, +ve, -ve
trim()	Removes leading/trailing spaces	" a ".trim() → "a"
replace()	"java".replace("a", "x")	"jxvx"
contains()	"hello".contains("ll")	true

✅ **2. StringBuilder vs StringBuffer**

Feature	StringBuilder	StringBuffer
Mutability	Mutable	Mutable
Thread Safe	❌ Not thread-safe	✅ Thread-safe (synchronized)
Performance	✅ Faster	❌ Slower
Introduced In	Java 5	Java 1.0
Package	java.lang	java.lang

🔸 **Example – StringBuilder:**

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" Java");
System.out.println(sb);  // Output: Hello Java
```

🔸 **Example – StringBuffer:**

```
StringBuffer sb = new StringBuffer("Thread");
sb.append(" Safe");
System.out.println(sb);  // Output: Thread Safe
```

✅ **3. String Pool vs Heap Memory**

🔹 **String Pool (SCP):**

- Part of **method area / metaspace**
- Holds **string literals** only
- Ensures **memory optimization**

```
String s1 = "java";
String s2 = "java";
System.out.println(s1 == s2); // ✅ true (same reference)
```

🔹 **Heap:**

- Stores objects created using new

```
String s1 = new String("java");
String s2 = new String("java");
System.out.println(s1 == s2); // ❌ false (new objects)
```

✅ **4. intern() Method**

🔹 **Use:**

Puts string into SCP and returns its reference.

```
String s1 = new String("core").intern();
String s2 = "core";
```

System.out.println(s1 == s2); // ✅ true

- 🔹 **Purpose:**
  - Save memory
  - Useful in large apps where string duplication is common

✅ 5. Summary Table

Class	Mutable	Thread Safe	Stored In	Performance
String	❌	✅ (Immutable)	SCP/Heap	Medium (new object each time)
StringBuilder	✅	❌	Heap	✅ Fast
StringBuffer	✅	✅	Heap	Slower

✅ 6. MCQ Tips

- 🔹 **String**
  1. "abc" == "abc" → ✅ true (both in SCP)
  2. new String("abc") == "abc" → ❌ false
  3. String is immutable → ✅ true
  4. "abc".charAt(3) → ❌ StringIndexOutOfBoundsException
- 🔹 **StringBuilder / Buffer**
  5. Which is faster? → ✅ StringBuilder
  6. Which is thread-safe? → ✅ StringBuffer
  7. Can StringBuilder be used in multithreaded env? → ❌ No
  8. append() is used in → ✅ StringBuilder and StringBuffer
- 🔹 **String Pool / intern()**
  9. What does intern() do?
     ✅ Moves object to String pool if not present
  10. "abc".intern() == "abc" → ✅ true

✅ Practice Code Snippet

```
public class Test {
  public static void main(String[] args) {
    String s1 = "java";
    String s2 = new String("java");
    String s3 = s2.intern();

    System.out.println(s1 == s2);  // false
    System.out.println(s1 == s3);  // true

    StringBuilder sb = new StringBuilder("Hello");
    sb.append(" World");
    System.out.println(sb);  // Hello World
  }
}
```

✅ **1. String in Java**
✅ **2. StringBuilder vs StringBuffer**
✅ **3. String Pool & Heap Memory**
✅ **4. Important Methods (intern(), equals(), ==, etc.)**
✅ **5. MCQ Tips + Code Snippets**

✅ **1. String Class in Java**

🔹 **Overview:**

- String is an **immutable** class in java.lang package.
- Stored in **String Constant Pool (SCP)**.
- Changing a String actually creates a **new object**.

```
String s1 = "hello";
s1 = s1.concat(" world");  // creates new string
System.out.println(s1);    // Output: hello world
```

🔹 **Why String is Immutable?**

- For security (passwords, file paths)
- Thread safety
- Hashcode consistency (used in keys of HashMap)
- SCP reuse

🔹 **Common String Methods:**

Method	Use Example	Output
length()	"abc".length()	3
charAt(int)	"abc".charAt(1)	'b'
substring(int)	"hello".substring(2)	"llo"
toUpperCase()	"hi".toUpperCase()	"HI"
equals()	"abc".equals("abc")	true
equalsIgnoreCase()	"Abc".equalsIgnoreCase("abc")	true
==	Compares references (not content)	false or true
compareTo()	Lexicographic compare	0, +ve, -ve
trim()	Removes leading/trailing spaces	" a ".trim() → "a"
replace()	"java".replace("a", "x")	"jxvx"
contains()	"hello".contains("ll")	true

✅ **2. StringBuilder vs StringBuffer**

Feature	StringBuilder	StringBuffer
Mutability	Mutable	Mutable
Thread Safe	❌ Not thread-safe	✅ Thread-safe (synchronized)
Performance	✅ Faster	❌ Slower
Introduced In	Java 5	Java 1.0

Feature	StringBuilder	StringBuffer
Package	java.lang	java.lang

🔸 **Example – StringBuilder:**

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" Java");
System.out.println(sb);  // Output: Hello Java
```

🔸 **Example – StringBuffer:**

```
StringBuffer sb = new StringBuffer("Thread");
sb.append(" Safe");
System.out.println(sb);  // Output: Thread Safe
```

✅ **3. String Pool vs Heap Memory**

🔹 **String Pool (SCP):**

- Part of **method area / metaspace**
- Holds **string literals** only
- Ensures **memory optimization**

```
String s1 = "java";
String s2 = "java";
System.out.println(s1 == s2); // ✅ true (same reference)
```

🔹 **Heap:**

- Stores objects created using new

```
String s1 = new String("java");
String s2 = new String("java");
System.out.println(s1 == s2); // ❌ false (new objects)
```

✅ **4. intern() Method**

🔹 **Use:**

Puts string into SCP and returns its reference.

```
String s1 = new String("core").intern();
String s2 = "core";

System.out.println(s1 == s2);  // ✅ true
```

🔹 **Purpose:**

- Save memory
- Useful in large apps where string duplication is common

✅ **5. Summary Table**

Class	Mutable	Thread Safe	Stored In	Performance
String	❌	✅ (Immutable)	SCP/Heap	Medium (new object each time)
StringBuilder	✅	❌	Heap	✅ Fast
StringBuffer	✅	✅	Heap	Slower

✅ **6. MCQ Tips**

🔹 **String**

1. "abc" == "abc" → ✅ true (both in SCP)
2. new String("abc") == "abc" → ❌ false
3. String is immutable → ✅ true
4. "abc".charAt(3) → ❌ StringIndexOutOfBoundsException

🔹 **StringBuilder / Buffer**

5. Which is faster? → ✅ StringBuilder
6. Which is thread-safe? → ✅ StringBuffer
7. Can StringBuilder be used in multithreaded env? → ❌ No
8. append() is used in → ✅ StringBuilder and StringBuffer

🔹 **String Pool / intern()**

9. What does intern() do?
   ✅ Moves object to String pool if not present
10. "abc".intern() == "abc" → ✅ true

✅ **Practice Code Snippet**

```
public class Test {
  public static void main(String[] args) {
    String s1 = "java";
    String s2 = new String("java");
    String s3 = s2.intern();

    System.out.println(s1 == s2);  // false
    System.out.println(s1 == s3);  // true

    StringBuilder sb = new StringBuilder("Hello");
    sb.append(" World");
    System.out.println(sb);  // Hello World
  }
}
```

✅ **What is the String Pool?**

- The **String Pool** is a **special memory area** inside the **Method Area / Metaspace** (older JVMs: PermGen).
- It stores **string literals** to save memory by **reusing existing objects**.
- This pool is also called the **String Constant Pool (SCP)**.

✅ **Object Creation for Strings in Java**

🔸 **1. Using String Literal**

```
String s1 = "hello";
String s2 = "hello";
```

- Only **one object** is created in the **String Pool**.
- s1 == s2 → ✅ true (same reference)

✅ **Efficient** and **recommended**

🔸 2. Using new keyword

String s1 = new String("hello");
String s2 = new String("hello");

- Each new creates a **new object in Heap**, **plus**:
  - "hello" is already in the **String Pool**
- s1 == s2 → ❌ false (different references)

🔸 3. Using intern() Method

String s1 = new String("hello");
String s2 = s1.intern(); // refers to SCP version
String s3 = "hello";

System.out.println(s2 == s3); // ✅ true

- intern() ensures the object is from **String Pool**
- Saves memory, especially in large-scale applications

✅ Visual Diagram:

✅ Summary Table

Statement	Where Stored	Reused?	Reference Equal (==)
String s = "java";	SCP	✅ Yes	Yes if same literal
String s = new String("java");	Heap (+ SCP)	❌ No	No
s.intern()	SCP	✅ Yes	Yes (if same literal)

✅ Important String Behaviors

String a = "hello";
String b = "hello";
System.out.println(a == b); // ✅ true (SCP)

String x = new String("world");
String y = new String("world");
System.out.println(x == y); // ❌ false (Heap)

String z = x.intern();
System.out.println(z == "world"); // ✅ true (SCP)

🧠 **MCQ Tips:**

1. "java" == "java" → ✅ true (same pool reference)
2. new String("java") == "java" → ❌ false (heap vs pool)
3. intern() method → ✅ pulls object into SCP
4. String literals go to → ✅ String Constant Pool
5. String objects created using new → ❌ not pooled automatically
6. "abc" == new String("abc") → ❌ false

✅ **1. toString() Method**
✅ **2. intern() Method**
✅ **3. hashCode() & equals()**
✅ **4. Full List of Important String Methods**
✅ **5. MCQ Tips & Examples**

✅ **1. toString() Method**

🔹 **Purpose:**

Converts an object into a **String representation**.

🔹 **In String class:**

It's overridden to return the **actual string content**.

```
String s = "Hello";
System.out.println(s.toString());   // Output: Hello
```

🔹 **Custom class example:**

```
class Test {}
Test t = new Test();
System.out.println(t);  // Output: Test@hashcode
```

📌 In String class:

```
@Override
public String toString() {
   return this;
}
```

✅ **2. intern() Method**

🔹 **Purpose:**

Returns a **canonical representation** from the **String Constant Pool (SCP)**.

```
String s1 = new String("hello");
String s2 = s1.intern();
String s3 = "hello";

System.out.println(s2 == s3);  // ✅ true
```

✅ **Key Point:**

- **Reduces memory** usage
- Ensures only **one copy** of each string literal exists

✅ **3. hashCode() and equals()**

🔹 **equals():**

Compares **content** of the strings.
String s1 = new String("Java");
String s2 = new String("Java");

System.out.println(s1.equals(s2)); // ✅ true (content match)

🔹 **hashCode():**
Returns an int hash value based on string content.
System.out.println("Java".hashCode()); // Consistent hash
System.out.println("Java".equals("Java")); // ✅ true

✅ **Rule:**
If s1.equals(s2) is true, then s1.hashCode() == s2.hashCode() is also true.

✅ **4. Important String Methods (MCQ-Worthy)**

Method	Description	Example
length()	Returns number of characters	"abc".length() → 3
charAt(int)	Returns character at index	"abc".charAt(1) → 'b'
substring(int, int)	Returns substring from start to end index-1	"hello".substring(1,3) → "el"
equals()	Compares content	"abc".equals("abc") → true
equalsIgnoreCase()	Ignores case	"Abc".equalsIgnoreCase("abc")
compareTo()	Lexicographic compare	"a".compareTo("b") → -1
==	Compares references	"abc" == new String("abc") → ❌
toUpperCase()	Returns uppercase	"abc".toUpperCase() → "ABC"
toLowerCase()	Returns lowercase	"AbC".toLowerCase() → "abc"
indexOf()	Index of character or string	"java".indexOf('a') → 1
lastIndexOf()	Last occurrence	"javajava".lastIndexOf("a") → 7
replace()	Replaces characters	"java".replace('a','x') → "jxvx"
contains()	Checks substring	"hello".contains("ll") → true
startsWith()	Checks prefix	"abc".startsWith("a") → true
endsWith()	Checks suffix	"abc".endsWith("c") → true
split()	Splits string	"a,b".split(",") → [a, b]
trim()	Removes leading/trailing whitespace	" a ".trim() → "a"
isEmpty()	Checks if empty	"".isEmpty() → true
isBlank() (Java 11)	Empty or whitespace	" ".isBlank() → true
intern()	SCP optimization	"abc".intern()
valueOf()	Converts any type to string	String.valueOf(123) → "123"
join() (Java 8)	Join multiple strings	String.join("-", "a", "b") → "a-b"
repeat() (Java 11)	Repeats the string	"hi".repeat(3) → "hihihi"

✅ 5. MCQ Tips (Most Askable):

🔸 **== vs equals()**

- == → compares **reference**
- equals() → compares **content**

```
String a = "Java";
String b = new String("Java");
System.out.println(a == b);     // ❌ false
System.out.println(a.equals(b)); // ✅ true
```

🔸 **intern() behavior**

```
String x = new String("code");
String y = x.intern();
String z = "code";

System.out.println(y == z);  // ✅ true
```

🔸 **hashCode() with equals()**

- If equals() is true → hashCode() must match
- If hashCode() same → equals() **may or may not** be true

🔸 **toString() override**

- Default Object.toString() → ClassName@hashcode
- String.toString() → returns actual content

✅ Interview MCQs:

1. String s1 = "abc"; String s2 = "abc"; s1 == s2
   ✅ true
2. "abc".equals(new String("abc"))
   ✅ true
3. "abc".hashCode() == new String("abc").hashCode()
   ✅ true
4. new String("abc") == "abc"
   ❌ false
5. .intern() ensures string is from?
   ✅ String Pool
6. String.valueOf(10) returns?
   ✅ "10" (String)
7. "hi".repeat(3) (Java 11+)
   ✅ "hihihi"

✅ Final Recap

Method	Key Use
toString()	Convert object to string
intern()	Return SCP reference

Method	Key Use
hashCode()	Efficient hashing; same for equal strings
equals()	Content comparison
==	Reference comparison

Here's a **complete and exam-oriented guide** to **Java Access Specifiers (Modifiers)** with examples and MCQ tips:

✅ **Java Access Specifiers (Modifiers)**
In Java, **access specifiers** control the **visibility** or **access level** of **classes**, **methods**, **constructors**, and **variables**.

✅ **There are 4 Access Specifiers:**

Access Modifier	Within Class	Within Package	Outside Package (Subclass)	Outside Package
private	✅ Yes	❌ No	❌ No	❌ No
default *(no keyword)*	✅ Yes	✅ Yes	❌ No	❌ No
protected	✅ Yes	✅ Yes	✅ Yes	❌ No
public	✅ Yes	✅ Yes	✅ Yes	✅ Yes

🔹 **1. private**

- Accessible **only within the same class**
- Not visible to subclasses or outside classes
- Most restrictive

```
class A {
  private int data = 10;
  private void show() {
    System.out.println("Private method");
  }
}
```

✅ **Use case:** Hiding sensitive data (Encapsulation)

🔹 **2. default (No modifier)**

- Accessible **within the same package only**
- Also known as **package-private**
- Not visible outside the package

```
class A {
  void show() {  // default
    System.out.println("Default method");
  }
}
```

✅ **Use case:** Restrict access to classes within the same module/package.

🔹 **3. protected**

- Accessible:
  - Within the same package (like default)
  - Outside package but only in **subclasses**
- Cannot be used for top-level classes

```
class A {
  protected void display() {
    System.out.println("Protected method");
  }
}

class B extends A {
  void show() {
    display();  // allowed
  }
}
```

✅ **Use case:** Inheritance with controlled access

🔹 **4. public**

- Accessible **from anywhere**
- Least restrictive
- If a class is declared public, it must be in a file **with the same name**.

```
public class A {
  public void show() {
    System.out.println("Public method");
  }
}
```

✅ **Use case:** API classes, libraries

✅ **Access Specifier for Classes**

Class Type	Allowed Modifiers
Top-level	public, *default* only
Inner class	All 4 access modifiers

✅ **Practical Table**

Modifier	Class	Variable	Method	Constructor
private	❌	✅	✅	✅
default	✅	✅	✅	✅
protected	❌	✅	✅	✅
public	✅	✅	✅	✅

✅ **Code Example: All Modifiers**

```
package pkg1;

public class A {
  public int pub = 1;
  protected int prot = 2;
  int def = 3;        // default
  private int priv = 4;

  public void show() {
    System.out.println(pub);
    System.out.println(prot);
    System.out.println(def);
    System.out.println(priv);
  }
}
package pkg2;

import pkg1.A;

class B extends A {
  void accessTest() {
    System.out.println(pub);   // ✅ OK
    System.out.println(prot);  // ✅ OK (because B extends A)
    // System.out.println(def); // ❌ Not accessible
    // System.out.println(priv);// ❌ Not accessible
  }
}
```

🧠 **MCQ Tips:**

1. **Which modifier gives least access?**
   ✅ private
2. **Which modifier is accessible in subclass from other package?**
   ✅ protected
3. **Can you use protected for top-level class?**
   ❌ No
4. **Which modifier is default if not written?**
   ✅ Package-private
5. **Which modifiers can be applied to top-level class?**
   ✅ public and *default*
6. **If a class is declared public, where should it be placed?**
   ✅ In a file with the same name as class
7. **What is the difference between default and protected?**
   ✅ Protected allows subclass access outside package, default doesn't.

✅ **Summary**

Modifier	**Most Restricted ←——————→ Least Restricted**
private < default < protected < public	

✅ **1. Java Singleton Class (in depth)**
✅ **2. Enum in Java**
✅ **3. Enum Constructors**
✅ **4. Enum and String**
✅ **MCQ Tips + Code Examples**

✅ **1. Java Singleton Class – Detailed**
A **Singleton class** allows **only one instance** of the class to be created in the JVM.

🔹 **Why Singleton?**

- Managing shared resources: DB connection, logging, config files
- Saves memory by avoiding multiple objects

✅ **Rules to Create Singleton:**

1. Private constructor
2. Private static instance variable
3. Public static method to return the instance

🔸 **Example – Lazy Initialization Singleton:**

```
public class Singleton {
  private static Singleton instance;

  private Singleton() { } // private constructor

  public static Singleton getInstance() {
    if (instance == null)
      instance = new Singleton();
    return instance;
  }
}
```

🔸 Usage:

```
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();
System.out.println(s1 == s2);  // ✅ true
```

🔸 **Thread-safe Singleton (Double-Checked Locking):**

```
public class Singleton {
```

```
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized(Singleton.class) {
                if (instance == null)
                    instance = new Singleton();
            }
        }
        return instance;
    }
}
```

🔸 **Singleton Using Static Block:**

```
public class Singleton {
    private static Singleton instance;

    static {
        instance = new Singleton();
    }

    private Singleton() {}

    public static Singleton getInstance() {
        return instance;
    }
}
```

✅ **2. Singleton Using enum (BEST PRACTICE)**

🔹 **Why enum?**

- Thread-safe
- Serialization-proof
- Prevents reflection attack

```
public enum MySingleton {
    INSTANCE;

    public void showMessage() {
        System.out.println("Singleton using enum!");
    }
}
```

🔸 Usage:

```
MySingleton obj = MySingleton.INSTANCE;
obj.showMessage();
```

✅ **3. Enum in Java**

🔹 **What is Enum?**

An enum is a **special data type** that contains a fixed set of constants.

```
enum Day {
  MON, TUE, WED, THU, FRI
}
```

🔸 **Key Features:**

- Implicitly extends java.lang.Enum
- Cannot extend any other class
- Can implement interfaces
- Can have **constructors**, **methods**, and **fields**

✅ **4. Enum Constructors and Fields**

🔸 **Enum with Fields and Constructor:**

```
enum Level {
  LOW("Low Level"), MEDIUM("Medium"), HIGH("High");

  private String description;

  Level(String desc) {
    this.description = desc;
  }

  public String getDescription() {
    return description;
  }
}
```

🔸 **Usage:**

```
public class Test {
  public static void main(String[] args) {
    Level l = Level.HIGH;
    System.out.println(l.getDescription());  // Output: High
  }
}
```

🔸 **Enums & String:**

- Enum constants can be converted to/from strings.

```
Level level = Level.valueOf("HIGH");  // From String
System.out.println(level);          // To String: HIGH
```

🔹 **Looping Over Enums:**

```
for (Level l : Level.values()) {
  System.out.println(l + " → " + l.getDescription());
}
```

✅ **MCQ Tips (Exam-Focused):**

1. Can you create a singleton using enum?
   ✅ Yes, safest method
2. Which class do enums extend?
   ✅ java.lang.Enum
3. Are enum constructors public?
   ❌ No, always **private by default**
4. Can enum have methods and fields?
   ✅ Yes
5. Is enum type-safe?
   ✅ Yes (better than constants)
6. How many objects of a singleton class can be created?
   ✅ Only one
7. Can reflection break enum singleton?
   ❌ No, JVM restricts it
8. Can enums implement interfaces?
   ✅ Yes

✅ **Summary Table**

Feature	Singleton Class	Enum Singleton
Thread Safe	❌ (unless synchronized)	✅ Always
Prevent Reflection	❌ (needs extra code)	✅ JVM prevents it
Serialization Safe	❌ (needs readResolve)	✅ Always safe
Easy to Implement	✅	✅ (recommended from Java 5+)

✅ **static final vs Constants in Java**

In Java, a **constant** is usually declared using static final keywords — but **not all static final variables are considered constants** unless they're also:

- **primitive or immutable**, and
- **assigned a value at declaration**

✅ **1. What is static final?**

- static: Belongs to the **class** (not instance)
- final: Cannot be **reassigned** once initialized

🔸 Together: A **single, unchangeable variable** shared by all instances of a class.

🔸 **Example:**

```
public class MyClass {
   static final int MAX_USERS = 100;  // ✅ constant
   static final String GREETING = "Hello"; // ✅ constant
}
```

✅ **2. What is a constant?**

A **constant** in Java is typically:

- public static final
- Assigned at compile-time
- Not supposed to change ever

🔸 Example:

```
public class Constants {
  public static final double PI = 3.14159;
  public static final String COUNTRY = "India";
}
```

These are true **constants**. JVM **inlines** them at compile time.

✅ **Table: static final vs Constant**

Feature	static final	Constant (public static final)
Shared across instances	✅ Yes	✅ Yes
Value can't be changed	✅ Yes (after assignment)	✅ Yes
Compiler inlining	❌ Not guaranteed	✅ Happens with primitive/String
Must initialize immediately?	❌ No (can be done in static block)	✅ Yes (for constant inlining)
Visibility	Depends (can be private/protected)	✅ Usually public for constants
Example	static final int x = 5;	public static final int x = 5;

✅ **When is static final not a constant?**

If you assign an object reference or use dynamic values:

static final Date today = new Date(); // ❌ Not constant

Because the value isn't known at compile time.

✅ **MCQ Tips**

1. What does static final mean?
   ✅ One copy for all, can't be changed
2. Is static final always a constant?
   ❌ No, only if assigned a compile-time constant value
3. Can final variable be initialized in constructor?
   ✅ Yes, if it's not static
4. Will public static final String PI = "3.14"; be inlined?
   ✅ Yes
5. Which is true for constants in Java?
   ✅ Must be public static final and assigned compile-time value

---

✅ **Best Practice for Constants**

```
public class AppConstants {
   public static final String APP_NAME = "MyApp";
   public static final int MAX_LIMIT = 1000;
}
```

✅ Use all **uppercase** with **underscores**

✅ Declare constants in **interface** or **utility class**

✅ **Abstract Class vs Interface in Java**

Feature	Abstract Class	Interface
Keyword	abstract	interface
Can contain method body?	Yes (can have both abstract & non-abstract)	Since Java 8: Yes (default & static methods)
Can contain constructors?	✅ Yes	❌ No
Can contain fields/variables?	✅ Yes (can be non-static, non-final)	✅ Yes (but always public static final)
Multiple inheritance?	❌ No	✅ Yes (a class can implement multiple interfaces)
Access modifiers on methods	Can be private, protected, etc.	Methods are public by default
Extends/Implements?	Class **extends** abstract class	Class **implements** interface
Purpose	Partial implementation or common base class	Total abstraction or capability (contract)

🔍 **When to Use What?**

- Use **abstract class** when:
  - You want to share code among several closely related classes.
  - You need non-static, non-final fields.
  - You want to provide default behavior but allow some overrides.
- Use **interface** when:
  - You want to define a contract without any implementation (or minimal).
  - You want multiple classes (even unrelated) to implement the same set of behaviors.

🚀 **Since Java 8+ Enhancements**

Java Version	Interface Feature Added
Java 8	Default methods, static methods
Java 9	Private methods in interfaces

🎯 **MCQ Tips & Tricks**

🔥 **Common MCQ Points:**

1. **Interfaces can extend multiple interfaces.**
   - ✅ True
   - interface A {} interface B extends A {}
2. **Can a class extend an abstract class and implement interfaces?**
   - ✅ Yes
3. class MyClass extends MyAbstractClass implements MyInterface { }
4. **Interfaces cannot have constructors.**
   - ✅ True
5. **All variables in an interface are implicitly:**
   - public static final
6. **An abstract class can have:**
   - **Constructors**
   - **Instance variables**
   - **Concrete methods**
7. **Can interfaces have private methods?**
   - ✅ Yes, since Java 9 (but only to support code reuse in default/static methods inside interfaces)

🧠 **Quick Mnemonic:**

"ABC of Interface"

- **A**bstract methods (100% in Java 7)
- **B**y default, all methods are public
- **C**ontract-like behavior (multiple inheritance supported)

✨ **Example Code**

Abstract Class

```
abstract class Animal {
  String name;
  abstract void makeSound();
  void sleep() {
    System.out.println("Sleeping...");
  }
}
```

Interface

```
interface Flyable {
  int WING_COUNT = 2; // public static final
  void fly();       // public abstract
}
```

✅ **1. What are Wrapper Classes?**

✅ **2. Autoboxing & Unboxing**

✅ **3. Common Methods**

✅ **4. Memory & Caching Behavior**

✅ **5. Wrapper Class Objects Comparison**

✅ 6. MCQ Tips & Code Examples

✅ 1. What Are Wrapper Classes?

Java provides **wrapper classes** to **convert primitive types into objects**.

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

✅ They are part of java.lang package.

✅ 2. Why Wrapper Classes?

🔹 Required when:

- Working with **Collections** (List, Set, Map) — they can't store primitives
- **Serialization**, **Reflection**, **Generic Types**, etc.
- Converting between Strings and numbers

✅ 3. Autoboxing & Unboxing

🔹 **Autoboxing:**

Primitive → Wrapper (automatic)

```
int a = 10;
Integer obj = a;  // autoboxing
```

🔹 **Unboxing:**

Wrapper → Primitive (automatic)

```
Integer i = 20;
int val = i;  // unboxing
```

✅ 4. Common Wrapper Class Methods

Method	Description
parseInt(String) (Integer)	Converts string to int
valueOf(String)	Returns wrapper object from string
xxxValue()	Converts to other primitive types (intValue())
toString()	Converts wrapper to String
compareTo()	Compares values
equals()	Checks object equality
MAX_VALUE, MIN_VALUE	Constants for each wrapper class
isNaN()	Checks if Double/Float is NaN

🔹 **Examples:**

```
int x = Integer.parseInt("100");      // 100
Integer y = Integer.valueOf("123");   // Wrapper
String s = y.toString();              // "123"
```

✅ **5. Memory & Caching Behavior (MCQ-Worthy)**

🔹 **Integer Caching:**

Java caches Integer values from **-128 to 127**.

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b);  // ✅ true

Integer x = 200;
Integer y = 200;
System.out.println(x == y);  // ❌ false
```

== compares reference, not value

Use .equals() to compare values safely

✅ **6. Wrapper vs Primitive Comparison**

Feature	Primitive	Wrapper Class
Stored in	Stack	Heap
Can be null	❌ No	✅ Yes
Default value	0, false	null
Used in Generics	❌ No	✅ Yes

✅ **7. MCQ Tips (Frequently Asked)**

🔹 **Q1:** What is the wrapper class for int?

✅ Integer

🔹 **Q2:** What will be output?

```
Integer a = 128;
Integer b = 128;
System.out.println(a == b);
```

❌ false — outside cache range

🔹 **Q3:** What is Integer.valueOf("123")?

✅ Returns an Integer object

🔹 **Q4:** What happens with Integer.parseInt("abc")?

❌ Throws NumberFormatException

🔹 **Q5:** Can a wrapper class be null?

✅ Yes

🔹 **Q6:** What is Integer.MAX_VALUE?

✅ 2147483647

✅ **8. Summary Table**

Operation	Example
Autoboxing	Integer i = 10;
Unboxing	int a = i;
parseInt()	Integer.parseInt("123")
valueOf()	Integer.valueOf("123")
equals()	i1.equals(i2)
Reference equality	i1 == i2 (may be false)
Constants	Integer.MAX_VALUE, etc.

✅ **1. Widening in Java**

🔹 **Widening → Smaller primitive → Bigger primitive**

```
int a = 10;
long l = a;     // ✅ Widening (int → long)
float f = l;    // ✅ Widening (long → float)
```

✅ **No data loss, done automatically by compiler**

🔴 **Narrowing (opposite of widening)**

- Must use **explicit cast**

```
int x = (int) 100.5; // ❌ Narrowing: double → int
```

✅ **MCQ Tip:**

Q: What will happen if you pass int where long is expected?

✅ Widening — **valid**

✅ **2. Overloading with Autoboxing**

Java can **autobox** primitive types to wrapper classes in method overloading.

```
void show(Integer i) {
  System.out.println("Integer");
}

void show(int i) {
  System.out.println("int");
}

show(10);  // ✅ int (exact match preferred)
```

🔹 **Priority Order in Overloading:**

1. **Exact match** (int → int)
2. **Widening** (int → long)
3. **Autoboxing** (int → Integer)
4. **Var-args**

✅ **Example:**

```
void print(Object o) { System.out.println("Object"); }
void print(int... a) { System.out.println("var-args"); }

print(5);  // ✅ Object via autoboxing → Integer → Object
```

✅ **3. Overloading with Var-args**

🔹 **Var-args → Variable arguments (int... a)**

```
void test(int... x) {
  System.out.println("varargs");
}

void test(Integer x) {
  System.out.println("Integer");
}

test(10);  // ✅ Integer — exact match is preferred
```

✅ **Rule:**

If method overloading involves var-args, it is **always the last** priority.

✅ **4. Bridge Methods in Java (Compiler Generated)**

🔹 **Used internally to maintain polymorphism with generics + type erasure**

🔸 **Example:**

```
class Parent<T> {
  T get() { return null; }
}

class Child extends Parent<String> {
  String get() { return "Child"; }
}
```

Java compiler generates a **bridge method**:

```
// Synthetic bridge method created:
Object get() {
  return get();  // calls actual get()
}
```

✅ This ensures Child can override Parent<T> even after type erasure (since JVM doesn't know generics).

✅ **MCQ Tip:**

Q: What is the purpose of a **bridge method**?

✅ Maintain polymorphism after **type erasure**

✅ **5. Covariant Return Types**

🔹 **Java allows overriding a method with a subclass return type (from Java 5)**

```
class A {
  Number show() { return 10; }
}

class B extends A {
  Integer show() { return 20; } // ✅ Covariant return type
}
```

✅ Integer is a **subclass** of Number, so this is valid

❌ **Invalid Example:**

```
class A {
  Number show() { return 10; }
}

class B extends A {
  String show() { return "Hi"; } // ❌ Compile Error
}
```

🔹 Because String is not a subclass of Number

✅ **MCQ Tip:**

Q: Can you override a method and change return type?

✅ Yes, if **covariant** (subclass type)

✅ **Summary Table**

Feature	Description
Widening	Auto conversion from small → large primitive
Autoboxing	Auto convert primitive to wrapper
Var-args	Accepts multiple arguments as array
Overloading Priority	Exact > Widening > Boxing > Var-args
Bridge Methods	Compiler adds for generic type erasure support
Covariant Return Types	Subclass return type allowed in overriding

---

✅ **Mini Quiz (MCQ Practice):**

1. What is the order of preference in overloaded methods?
   ✅ Exact → Widening → Boxing → Varargs
2. Will int automatically be boxed to Integer?
   ✅ Yes
3. Is Integer to Object valid by overloading?
   ✅ Yes (autoboxing + upcast)
4. What is the purpose of bridge methods?
   ✅ Preserve method compatibility after type erasure

5. Can covariant return types be used with overriding?
   ✅ Yes, subtype only

✅ **Covariant Return Type – Simple Explanation:**
Covariant means **"allowing a more specific type"** when overriding a method.

🔹 **In Java:**
When you **override a method** in a child class, the **return type can be a subclass** of the parent method's return type.

🔸 **Example:**

```
class Animal { }

class Dog extends Animal { }

class Parent {
  Animal getPet() {
    return new Animal();
  }
}

class Child extends Parent {
  Dog getPet() { // ✅ Covariant return type
    return new Dog();
  }
}
```

🔹 Here, Dog is a **covariant return** because it's a **subtype** of Animal.

🧠 **One-line Trick:**
✅ "Child method can return **subclass** of parent's return type."

❌ **Not Allowed:**
Changing return type to **unrelated class**:

```
// Invalid override:
String getPet() ❌ // Not a subclass of Animal
```

✅ 1. What is Exception Handling?

Java **Exception Handling** is used to **handle runtime problems** like divide by zero, file not found, null pointer, etc., **gracefully** (without program crash).

- 🔹 **Exception = Runtime issue that can be handled**
- 🔹 **Error = Serious issue that cannot/should not be handled (e.g., OutOfMemoryError)**

✅ 2. Exception vs Error

Feature	Exception	Error
Type	Problems during runtime	Serious issues (JVM-related)
Can be handled?	✅ Yes	❌ No (generally not)
Example	NullPointerException, IOException	StackOverflowError, OutOfMemoryError

---

✅ 3. Checked vs Unchecked Exceptions

Feature	Checked Exception	Unchecked Exception
Compile-time check	✅ Required	❌ Not required
Extends	Exception	RuntimeException
Must declare with throws?	✅ Yes	❌ No
Examples	IOException, SQLException	NullPointerException, ArithmeticException

---

- 🔹 **Checked = must handle**

```
public void readFile() throws IOException {
  FileReader fr = new FileReader("file.txt");
}
```

- 🔹 **Unchecked = optional handling**

```
int a = 10 / 0;  // ArithmeticException
```

✅ 4. RuntimeException vs IOException

RuntimeException	IOException
Unchecked	Checked
Caused by code issues	Caused by external resources
Example: divide by 0	File not found, stream error

✅ 5. try-catch-finally

```
try {
  // code that may throw exception
} catch (Exception e) {
  // handle exception
} finally {
  // cleanup code
}
```

✅ When does finally not execute?

- When JVM shuts down (System.exit(0))
- If program **crashes hard** (like a kill signal)

✅ Otherwise, **finally block always runs**

✅ 6. throw vs throws

Keyword	Purpose	Used With
throw	**Actually throws** an exception	Inside method
throws	**Declares** exception	In method signature

🔹 **throw Example:**

```
throw new ArithmeticException("Divide by zero");
```

🔹 **throws Example:**

```
public void read() throws IOException { ... }
```

✅ MCQ Tip:

throw = raise
throws = declare

✅ 7. Common Exceptions (with package)

Exception	Type	Notes
NullPointerException	Unchecked	Accessing object with null
ArrayIndexOutOfBounds	Unchecked	Invalid array index
ArithmeticException	Unchecked	Divide by zero
NumberFormatException	Unchecked	Parsing non-numeric string
ClassNotFoundException	Checked	Class not found in classpath
IOException	Checked	I/O related issues
FileNotFoundException	Checked	File not found
SQLException	Checked	DB errors
IllegalArgumentException	Unchecked	Wrong argument passed to method
InterruptedException	Checked	Thread interruption

✅ 8. Custom Exception (Optional)

```
class MyException extends Exception {
  MyException(String msg) {
    super(msg);
  }
}
```

✅ 9. Summary Table

Concept	Example	MCQ Tip
Checked Exception	IOException	Must use throws or try-catch
Unchecked Exception	NullPointerException	No handling needed (optional)
throw	throw new XException()	Used to raise exception
throws	void m() throws IOException	Used to declare exception
finally	Cleanup block	Always runs (except System.exit())
Error	OutOfMemoryError	Cannot be handled by application

🧠 MCQ Tips

1. Which is a checked exception?
   ✅ IOException
2. Can finally block run without catch?
   ✅ Yes
3. Which of the following will **not compile**?
4. public void m() {
5. throw new IOException(); // ❌ must declare throws
6. }
7. Which is true about throw and throws?
   ✅ throw is to raise, throws is to declare
8. What happens when exception not caught?
   ✅ JVM terminates program and prints stack trace

✅ **Java Enum Basics**

```
enum Day {
   MONDAY, TUESDAY, WEDNESDAY;
}
```

This enum defines 3 constants. Internally, these have:

Enum Constant	ordinal()	name()	toString()
MONDAY	0	"MONDAY"	"MONDAY"
TUESDAY	1	"TUESDAY"	"TUESDAY"
WEDNESDAY	2	"WEDNESDAY"	"WEDNESDAY"

🔍 **1. ordinal()**

- Returns the **index** of the enum constant.
- Example:
- Day d = Day.TUESDAY;
- System.out.println(d.ordinal()); // Output: 1

🔍 **2. name() and toString()**

- name() → returns the exact enum constant name as declared.
- toString() → same as name() unless overridden.

```
System.out.println(d.name());     // "TUESDAY"
System.out.println(d.toString()); // "TUESDAY"
```

You can override toString() for custom output.

🔁 **3. values()**

- Returns all constants in an array, in declared order.

```
Day[] days = Day.values();
System.out.println(days[0]); // MONDAY
System.out.println(days[1]); // TUESDAY
```

🧠 ✅ **Convert ordinal to String Using values()**

Let's say you have an **ordinal (int)** and want the **enum name**:

```
int ordinal = 2;
String dayName = Day.values()[ordinal].name();
System.out.println(dayName); // WEDNESDAY
```

Or just:

```
String dayString = Day.values()[ordinal].toString();
```

⚠️ **Important Note:**

Always check bounds to avoid ArrayIndexOutOfBoundsException:

```
if (ordinal >= 0 && ordinal < Day.values().length) {
   String day = Day.values()[ordinal].name();
}
```

📌 **Bonus: Override toString() Example**

```
enum Day {
  MONDAY, TUESDAY, WEDNESDAY;

  @Override
  public String toString() {
    switch(this) {
      case MONDAY: return "First day";
      case TUESDAY: return "Second day";
      case WEDNESDAY: return "Third day";
      default: return "Unknown";
    }
  }
}
```

Now Day.MONDAY.toString() → "First day"

✅ 1. What is Cloning in Java?

Cloning in Java means **creating an exact copy of an object in memory**.

- 🔹 Java provides a **clone() method** in the Object class
- 🔹 Used to create a **duplicate object without using new keyword**

✅ 2. Why Do We Need clone()?

- To **copy the current object's state** into a new object.
- Saves time compared to creating and setting values manually.
- Used in **design patterns** like Prototype, or when handling mutable objects.

✅ 3. clone() Method — Syntax

```
protected Object clone() throws CloneNotSupportedException
```

- It is protected in Object class
- Returns a **shallow copy** by default
- Needs:
  - implements Cloneable
  - override clone() as public

✅ 4. How to Use clone()

🔹 **Step-by-Step:**

```
class Student implements Cloneable {
  int id;
  String name;

  Student(int id, String name) {
    this.id = id;
    this.name = name;
  }

  public Object clone() throws CloneNotSupportedException {
    return super.clone();  // calls Object.clone()
  }
}
```

🔹 **Usage:**

```
Student s1 = new Student(1, "Yash");
Student s2 = (Student) s1.clone();
```

✅ 5. Important Points:

Point	Details
clone() is in Object class	Returns **Object**, must cast
protected method	So, must override as public to use externally
Must implement Cloneable	Else, CloneNotSupportedException is thrown
Shallow copy by default	Copies primitive & references (not deep)

✅ 6. Shallow vs Deep Cloning

🔹 **Shallow Copy:**

- Copies **object reference**, not inner objects
- Changes in original affect clone

```
class A implements Cloneable {
  int[] data = {1, 2};

  public Object clone() throws CloneNotSupportedException {
    return super.clone();
  }
}
```

🔹 **Deep Copy:**

- You **manually clone inner objects**
- Each object has **separate memory**

```
public Object clone() throws CloneNotSupportedException {
  A cloned = (A) super.clone();
  cloned.data = data.clone();  // clone inner object
  return cloned;
}
```

✅ **7. Customizing clone()**

```
@Override
public Student clone() {
  try {
    return (Student) super.clone();
  } catch (CloneNotSupportedException e) {
    throw new AssertionError();  // won't occur if Cloneable is implemented
  }
}
```

✅ **8. MCQ Tips & Common Questions**

1. **Is clone() a public method in Object?**
   ❌ No, it is protected
2. **What happens if Cloneable is not implemented?**
   ❌ CloneNotSupportedException is thrown
3. **Is clone() deep or shallow?**
   ✅ Shallow by default
4. **How to make deep copy?**
   ✅ Manually clone inner objects or use serialization
5. **Why cast is needed for clone()?**
   ✅ clone() returns Object
6. **Does clone() call constructor?**
   ❌ No, it **copies memory**, does not invoke constructor

✅ **9. Real-Life Use Cases**

- Creating backups of objects
- Undo operations in editors

- Prototyping or design patterns like Prototype Pattern
- Cloning configurations/templates

✅ **10. Summary Table**

Concept	Explanation
Method location	Object class
Signature	protected Object clone()
Exception	CloneNotSupportedException
Required interface	Cloneable (marker interface)
Default behavior	Shallow copy
Deep copy?	Not supported by default, must implement
Constructor called?	❌ No

🔷 **1. Interface Types**

✅ **A. Marker Interface**

- An interface with **no methods**.
- Used to "mark" a class for **special behavior** by the JVM or libraries.
- Examples:
  - Cloneable – tells JVM the object can be cloned.
  - Serializable – tells JVM the object can be serialized.
  - Remote – marks classes for RMI.

```
class MyClass implements Serializable { }
```

✅ **B. Functional Interface**

- Interface with **only one abstract method** (SAM – Single Abstract Method).
- Used for **lambda expressions** (Java 8+).
- Examples:
  - Runnable, Comparable, Callable, Predicate<T>, Consumer<T>

```
@FunctionalInterface
interface MyFunc {
  void doSomething();
}
```

✅ **C. Normal Interface**

- Defines a **contract**.
- Can have default, static, abstract, and (Java 9+) private methods.
- Can be extended by another interface or implemented by classes.

🔷 **2. Class Types**

✅ **A. Concrete Class**

- A regular class with full implementation.
- Can be instantiated directly.

```
class Car {
  void drive() { } }
```

✅ B. Abstract Class

- Cannot be instantiated.
- Can have both abstract and concrete methods.
- Used when you want to **partially define behavior**.

```
abstract class Animal {
  abstract void sound();
  void sleep() { System.out.println("Sleeping"); }
}
```

✅ C. Final Class

- Cannot be extended/inherited.
- Example: String is a final class.

```
final class Constants { }
```

✅ D. Static Nested Class / Inner Class / Local / Anonymous

Type	Description
Static Nested	Nested class with static, no outer ref
Inner	Non-static class tied to outer instance
Local	Defined inside a method
Anonymous	No name, used for one-time usage (esp. interface implementations)

```
class Outer {
  class Inner {}
  static class StaticNested {}
}
```

🔷 3. Important Class APIs

✅ A. Class<?> object

- Every class in Java has an associated Class object.

```
Class<?> cls = String.class;
System.out.println(cls.getName()); // java.lang.String
```

✅ B. Reflection

- Allows you to inspect classes, fields, methods at runtime.

```
Class<?> c = Class.forName("java.lang.String");
Method[] methods = c.getDeclaredMethods();
```

🔷 4. Object-Oriented Concepts in Java

Concept	Explanation	Example
Encapsulation	Hiding internal details using access modifiers	private fields + getters/setters
Inheritance	Acquiring properties from a superclass	class Car extends Vehicle
Polymorphism	Same method behaves differently	Method Overriding, Overloading

---

In Java, Class<?> is a generic type. Here's what each part means:

Class is a class in the Java Reflection API that represents classes and interfaces in the JVM.

<?> is a wildcard that means “unknown type.”

So, Class<?> means:

"This is a Class object, but we don't know (or care) which specific type it represents."

This is super useful when you're writing code that works with different types, but doesn’t need to know what those types are.

Concept	Explanation	Example
Abstraction	Hiding the complex logic, showing essentials	Interfaces, Abstract Classes

---

🔷 5. Essential Interfaces & Classes in Java API

Interface / Class	Purpose
Comparable<T>	Natural ordering (used in Collections.sort)
Comparator<T>	Custom sorting logic
Iterable<T>	Allows use in enhanced for-loop
Runnable	Used to define tasks for threads
Callable<V>	Like Runnable but returns a result
AutoCloseable	Used in try-with-resources
Serializable	Marks object to be serialized
Cloneable	Enables object cloning

🔷 6. Common Java Keywords

Keyword	Usage
final	Constant, non-overridable, non-extendable
static	Shared among all instances
this	Refers to current object
super	Refers to superclass
transient	Skip during serialization
volatile	Used in multithreading
synchronized	Used to control thread access
instanceof	Checks object type at runtime

🔷 7. Java Memory + Object Lifecycle Concepts

- **Heap** – where objects live
- **Stack** – where methods and local variables live
- **GC (Garbage Collector)** – automatically frees memory
- **finalize()** (deprecated) – cleanup before GC

🔷 8. Common Design Patterns in Java

Pattern	Use Case	Example
Singleton	One instance only	Runtime.getRuntime()
Factory	Creates objects based on input	JDBC Drivers
Builder	Step-by-step object creation	StringBuilder
Observer	Notify observers on change	Event Listeners
Strategy	Runtime selection of algorithms	Comparator/Sorters
Decorator	Add behavior dynamically	Streams in Java IO

🔷 9. Java 8+ Core Features (Must-Know)

Feature	Description
Lambda Expressions	Compact way to implement functional interfaces
Streams API	Functional-style operations on collections
Optional	Avoid null checks
Method References	System.out::println
Default/Static in Interface	Methods with implementation in interface

---

🎓 Summary Mindset: Learn by Category

Category	Learn About
Types of Classes	abstract, final, concrete, inner
Types of Interfaces	marker, functional, regular
Reflection & Class API	Class, Method, Field
Core Java Patterns	Singleton, Factory, Observer
Modern Java (8+)	Lambda, Stream, Functional Interfaces
JVM Concepts	GC, Heap, ClassLoader

✅ **1. What is Immutability?**

An immutable object is an object whose state (data) cannot change after it is created.

📌 Once created, its **field values cannot be modified**

📌 **String**, **Integer**, and **all wrapper classes** in Java are **immutable**

✅ **2. Why Immutability?**

🔹 Safer in **multi-threading**

🔹 Improves **security** (no external modification)

🔹 Easy to **cache and reuse** (like in String Pool)

🔹 Helps in **functional programming** (no side effects)

✅ **3. How to Create an Immutable Class?**

🔒 **Rules to Make a Class Immutable:**

Rule #	Description
1	Make class final → So it can't be extended
2	Make all fields private and final
3	No setters
4	Initialize fields via constructor only
5	If mutable object used (e.g., Date, List), return a **defensive copy** from getter

---

🔹 **Example:**

```
public final class Student {
  private final int id;
  private final String name;

  public Student(int id, String name) {
    this.id = id;
    this.name = name;
  }

  public int getId() { return id; }
  public String getName() { return name; }
}
```

✅ No way to modify id or name after object is created.

🔹 **If using mutable field (like Date):**

```
public final class Employee {
  private final Date joinDate;

  public Employee(Date date) {
    this.joinDate = new Date(date.getTime()); // defensive copy
  }
```

```
  public Date getJoinDate() {
    return new Date(joinDate.getTime());  // defensive copy
  }
}
```

✅ **4. Examples of Immutable Classes in Java:**

Class	Immutable?
String	✅ Yes
Integer	✅ Yes
LocalDate	✅ Yes
StringBuilder	❌ No
ArrayList	❌ No

✅ **5. Immutable vs Mutable**

Feature	Immutable	Mutable
Fields can change?	❌ No	✅ Yes
Thread-safe?	✅ Yes	❌ No (unless synchronized)
Examples	String, Integer	StringBuilder, ArrayList

✅ **6. Why is String Immutable?**

- Stored in **String Pool** for memory optimization
- Safe from **modification by multiple threads**
- Used as **keys in Map**, so hashcode should not change
- Helps in **caching and security**

🔸 **Example:**

```
String s = "abc";
s.concat("xyz");  // new object created
System.out.println(s);  // abc
```

✅ **7. MCQ Tips**

1. Which of the following are immutable in Java?
   ✅ String, Integer, LocalDate
2. Can we extend an immutable class?
   ❌ No (must be final)
3. Why is immutability good in multi-threaded apps?
   ✅ No thread can modify object state
4. Can you make an immutable class with a mutable field?
   ✅ Yes, with **defensive copies**
5. Can the clone() method break immutability?
   ✅ Yes, if deep copy is not used

✅ 8. Summary Diagram

Immutability
├── Final class
├── Final fields
├── No setters
├── Constructor-based init
└── Defensive copy for mutable fields

✅ 1. What is Java Reflection?

Reflection is a feature in Java that allows **inspection and modification** of classes, methods, fields, and constructors **at runtime**, even if they are private.

✅ Use Cases:

- Reading private fields/methods
- Creating objects dynamically
- Calling methods dynamically
- Used in **frameworks**: Spring, Hibernate, JUnit
- Serialization, Deserialization

✅ 2. Packages Involved

All reflection classes are in the **java.lang.reflect** package.
Also use:
java.lang.Class
java.lang.reflect.Field
java.lang.reflect.Method
java.lang.reflect.Constructor

✅ 3. Getting the Class Object

Three Ways to get a Class object:

```
Class<?> c1 = Class.forName("com.example.MyClass");  // Most common
Class<?> c2 = MyClass.class;
MyClass obj = new MyClass();
Class<?> c3 = obj.getClass();
```

✅ 4. Important Methods in Reflection API

Class API	Method	Use
Class	getName()	Get full class name
	getDeclaredFields()	Get all fields (including private)
	getDeclaredMethods()	Get all methods
	getDeclaredConstructors()	Get all constructors
	newInstance() (deprecated)	Create object (now use Constructor.newInstance())
Field	get(Object obj)	Get field value
	set(Object obj, Object value)	Set field value

Class API	Method	Use
	setAccessible(true)	Allow access to private field
Method	invoke(Object obj, Object... args)	Call method dynamically
Constructor	newInstance(Object... args)	Create object with constructor

---

✅ **5. Example: Access Private Field and Method**

```
import java.lang.reflect.*;

class Person {
  private String name = "Yash";

  private void greet() {
    System.out.println("Hello " + name);
  }
}

public class ReflectDemo {
  public static void main(String[] args) throws Exception {
    Person p = new Person();
    Class<?> c = p.getClass();

    // Access private field
    Field f = c.getDeclaredField("name");
    f.setAccessible(true);
    f.set(p, "Amit");

    // Access private method
    Method m = c.getDeclaredMethod("greet");
    m.setAccessible(true);
    m.invoke(p);  // prints: Hello Amit
  }
}
```

✅ **6. Reflection and Performance**

🔴 Reflection is **slower** than direct access

🔴 May **break encapsulation**

🔴 Can throw exceptions like:

- ClassNotFoundException
- NoSuchFieldException
- IllegalAccessException
- InvocationTargetException

✅ **7. Reflection and Constructors**

```
Constructor<?> cons = Class.forName("Person").getConstructor();
Object obj = cons.newInstance();
```

✅ **8. Reflection and Arrays (optional advanced)**

Use java.lang.reflect.Array to create or access arrays dynamically.

```
int[] arr = (int[]) Array.newInstance(int.class, 5);
Array.set(arr, 0, 100);
System.out.println(Array.get(arr, 0));  // 100
```

✅ **9. MCQ Tips**

Q1: What does Class.forName("A") return?

✅ Class object of class A

Q2: Can private methods be called using reflection?

✅ Yes, using setAccessible(true)

Q3: What will getClass() return?

✅ Returns the runtime class of the object

Q4: How to get all declared methods including private?

✅ getDeclaredMethods()

Q5: What is use of invoke() in reflection?

✅ Used to call a method dynamically

Q6: Reflection API breaks which OOP principle?

✅ Encapsulation

Q7: Which exception is thrown when class is not found?

✅ ClassNotFoundException

Q8: Which method is used to change a private field's value?

✅ set(Object obj, Object value)

✅ **10. Summary Table**

Concept	Method / Class
Get class info	Class.forName(), getClass()
Access private field	getDeclaredField(), setAccessible()
Call method	getDeclaredMethod(), invoke()
Create object dynamically	Constructor.newInstance()
Bypass access modifiers	setAccessible(true)

✅ 1. Ways to Break OOP Concepts in Java

Object-Oriented Programming (OOP) principles can be **violated** or **bypassed** using special features or poor practices. Let's break them down:

🔹 **OOP Principles:**

1. **Encapsulation**
2. **Abstraction**
3. **Inheritance**
4. **Polymorphism**

✅ **Ways to Break OOP Principles:**

OOP Principle	Can be Broken Using	How It's Broken
Encapsulation	✅ **Reflection API**	Can access and modify private fields/methods using setAccessible(true)
Abstraction	❌ Normally not breakable	But poorly designed interfaces/classes may expose too much
Inheritance	✅ Improper Design	Tight coupling, violating "Liskov Substitution" principle
Polymorphism	✅ Type Casting Abuse	Using explicit downcasting leads to ClassCastException
Immutability	✅ Cloning or Reflection	Can mutate supposedly immutable classes via reference fields or reflection

---

✅ **Example: Breaking Encapsulation**

```
Field f = Person.class.getDeclaredField("name");
f.setAccessible(true);  // ❌ breaks encapsulation
f.set(personObj, "Hacked");
```

✅ **MCQ Tip:**

Q: Which Java feature can be used to access private fields?

✅ Reflection (via setAccessible(true))

✅ 2. Ways to Create Objects in Java

Java allows object creation in **multiple ways**. Knowing these helps in understanding **design patterns**, **JVM internals**, and **interview MCQs**.

✅ **All Valid Ways to Create Objects in Java:**

Way	Syntax Example	Notes
1. **Using new keyword**	Person p = new Person();	Most common

Way	Syntax Example	Notes
2. **Using clone()**	Person p2 = (Person) p1.clone();	Class must implement Cloneable
3. **Using Class.forName() and Reflection**	Class.forName("Person").newInstance();	Deprecated. Use Constructor object
4. **Using Constructor class (Reflection)**	Constructor<Person> c = Person.class.getConstructor();Person p = c.newInstance();	Safer than Class.newInstance()
5. **Using Deserialization**	ObjectInputStream ois = new ObjectInputStream(new FileInputStream("obj.ser"));Person p = (Person) ois.readObject();	Reconstructs object from byte stream
6. **Using Factory Method**	Person p = PersonFactory.createPerson();	Useful in design patterns
7. **Using newInstance() of ClassLoader**	ClassLoader cl = MyClass.class.getClassLoader();Class<?> cls = cl.loadClass("MyClass");Object obj = cls.newInstance();	Advanced reflection
8. **Using Lambda or Method Reference**	Supplier<Person> s = Person::new;Person p = s.get();	Java 8 style object creation

---

✅ **Bonus: Prototype Design Pattern**

- Uses clone() method to create object copies

🔹 **Summary Table**

Method	Object Creation Style	When to Use
new	Normal constructor call	Regular use
clone()	Copy object	Requires Cloneable, shallow copy
Class.newInstance()	Reflection-based	Deprecated in Java 9+
Constructor.newInstance()	Better reflection method	Use when class name is dynamic
Deserialization	From file/stream	Object persistence
Factory method	Custom method	Encapsulation, flexibility
ClassLoader	Dynamic class loading	Advanced frameworks (Spring, etc.)

✅ **MCQ Tips**

Q1: Which of the following can create object without using new?

✅ clone(), deserialization, reflection

Q2: Which interface is required for cloning?

✅ Cloneable

Q3: Can deserialization call constructor?

❌ No. Deserialization bypasses constructor

Q4: Can reflection break encapsulation?

✅ Yes

Q5: What happens if Cloneable is not implemented?

❌ CloneNotSupportedException

🧠 **Flash Revision**

🔹 **OOP Principle Breakers:**

- Reflection → Breaks encapsulation
- Downcasting → Breaks polymorphism safely
- Misusing inheritance → Breaks Liskov's Principle

🔹 **Object Creation Techniques:**

new | clone() | Class.forName() | Constructor.newInstance()
Deserialization | Factory method | ClassLoader | Lambda

✅ **1. Java Executors Framework (Multithreading)**

🔹 **What is it?**

The **Executor framework** in Java is a **high-level API** to manage **thread pools** and **task execution** efficiently without manually creating threads.
It provides a **better alternative to using Thread and Runnable directly**.

✅ **1.1 Interfaces in Executor Framework:**

Interface	Description
Executor	Base interface with execute(Runnable)
ExecutorService	Extends Executor, adds lifecycle control
ScheduledExecutorService	Allows task scheduling (e.g., delay, periodic)

✅ **1.2 Common Implementations:**

Class	Description
Executors.newFixedThreadPool(n)	Fixed number of threads
Executors.newCachedThreadPool()	Unlimited threads, reuse idle
Executors.newSingleThreadExecutor()	One thread

Class	Description
Executors.newScheduledThreadPool(n)	Schedule tasks with delay or periodically

---

✅ **1.3 Example: Fixed Thread Pool**

```
ExecutorService service = Executors.newFixedThreadPool(2);

Runnable task = () -> {
  System.out.println("Task: " + Thread.currentThread().getName());
};

service.submit(task);
service.shutdown();
```

🔹 **Methods of ExecutorService**

Method	Description
submit()	Submits task and returns Future
shutdown()	Graceful shutdown (no new tasks)
shutdownNow()	Immediate shutdown
invokeAll()	Executes collection of Callables
invokeAny()	Executes and returns result of any one

🔹 **MCQ Tip:**

Q: Which method submits a task and returns result in future?

✅ submit() returns a Future<?>

✅ **2. ReentrantLock in Java**

🔹 **What is ReentrantLock?**

ReentrantLock is a **class from java.util.concurrent.locks** package that provides **explicit lock mechanisms** for threads.
It offers more control compared to synchronized.

✅ **Key Features of ReentrantLock:**

Feature	Description
Reentrancy	A thread can acquire the same lock multiple times
tryLock()	Try to acquire lock without blocking
lockInterruptibly()	Allows a thread to be interrupted while waiting for a lock
fair locking	Acquire lock in order of request (optional)

✅ **Basic Example:**

```
ReentrantLock lock = new ReentrantLock();

lock.lock();
try {
```

```
    // critical section
    System.out.println("Locked by: " + Thread.currentThread().getName());
} finally {
    lock.unlock();
}
```

✅ **tryLock() Example:**

```
if (lock.tryLock()) {
    try {
        // work
    } finally {
        lock.unlock();
    }
} else {
    System.out.println("Lock not available");
}
```

✅ **Reentrancy Explained:**

```
lock.lock();
lock.lock();  // same thread can reacquire
try {
  // logic
} finally {
  lock.unlock();  // must be called twice!
  lock.unlock();
}
```

✅ **ReentrantLock vs synchronized**

Feature	synchronized	ReentrantLock
Lock release	Auto-release	Manual unlock() required
Try lock	❌ Not available	✅ Yes (tryLock())
Interruptible	❌ No	✅ Yes (lockInterruptibly())
Fairness option	❌ No	✅ Yes (FIFO with constructor)
Condition support	❌ No	✅ Yes (Condition objects)

✅ **ReentrantLock Constructor:**

```
ReentrantLock lock = new ReentrantLock(true); // fair locking
```

✅ **MCQ Tips**

Q1: Which of the following returns a Future?

✅ submit()

Q2: What does shutdownNow() do?

✅ Attempts to stop all running tasks immediately

Q3: Which lock allows fair thread ordering?

✅ ReentrantLock(true)

Q4: What does tryLock() return?

✅ true if lock acquired, else false

Q5: What happens if unlock() is not called?

❌ Potential deadlock – always use finally

✅ **Summary: Executors vs ReentrantLock**

Feature	Executors	ReentrantLock
Used For	Managing threads & tasks	Controlling thread access
Introduced In	Java 5	Java 5
Benefit	Thread pool management	Fine-grained locking
Key Interface	ExecutorService	Lock

✅ **1. What is a Thread?**

A **Thread** is a lightweight sub-process. It's the smallest unit of execution.

- In Java, a thread is represented by the **java.lang.Thread** class.
- Multithreading is the ability of a program to execute **multiple threads concurrently**.

✅ **2. Process vs Thread**

Feature	Process	Thread
Definition	Independent program in execution	Lightweight unit of execution
Memory	Separate memory	Shared memory
Communication	Difficult	Easy
Overhead	High	Low

✅ **3. Ways to Create Threads in Java**

🔹 **Method 1: Extending Thread class**

```
class MyThread extends Thread {
  public void run() {
    System.out.println("Running thread");
  }
}
MyThread t = new MyThread();
t.start();  // start thread
```

🔹 **Method 2: Implementing Runnable interface**

```
class MyRunnable implements Runnable {
  public void run() {
    System.out.println("Runnable thread");
  }
}
```

```
Thread t = new Thread(new MyRunnable());
t.start();
```

🔹 **Method 3: Using ExecutorService (Thread Pool)**

```
ExecutorService es = Executors.newFixedThreadPool(3);
es.submit(() -> System.out.println("From thread pool"));
es.shutdown();
```

✅ **4. Lifecycle of a Thread**

```
NEW → RUNNABLE → RUNNING → TERMINATED
        ↓
   BLOCKED / WAITING / TIMED_WAITING
```

States:

- **NEW** – Thread created but not started
- **RUNNABLE** – Ready to run (in thread scheduler)
- **RUNNING** – Actively executing
- **BLOCKED** – Waiting for a monitor lock
- **WAITING / TIMED_WAITING** – Thread waiting indefinitely or for a specific time
- **TERMINATED** – Execution complete or stopped

✅ **5. Important Thread Methods**

Method	Purpose
start()	Starts the thread (calls run())
run()	Defines the code for the thread
sleep(ms)	Pauses the thread for milliseconds
join()	Waits for thread to finish
yield()	Hints to let other thread run
isAlive()	Checks if thread is alive
interrupt()	Interrupts a sleeping thread

✅ **6. Synchronization (Avoid Race Conditions)**

Race Condition = Two threads accessing shared data at the same time

🔹 **Synchronized Method**

```
public synchronized void increment() { count++; }
```

🔹 **Synchronized Block**

```
synchronized(this) {
   count++;
}
```

✅ **7. Thread Safety & Deadlocks**

- **Thread-safe**: Code that handles shared data correctly when accessed by multiple threads
- **Deadlock**: Two threads waiting for each other's lock → stuck forever

🔹 **Deadlock Example:**

```
synchronized(obj1) {
```

```
    synchronized(obj2) { ... } // Deadlock if another thread does reverse
}
```

✅ **8. Thread Priorities**

- Range: Thread.MIN_PRIORITY (1) to Thread.MAX_PRIORITY (10)
- Default: Thread.NORM_PRIORITY (5)
- Higher priority ≠ guaranteed execution order (depends on JVM)

✅ **9. Daemon Threads**

- Background threads that die when main thread dies

```
t.setDaemon(true);  // before start()
```

Examples: Garbage Collector, JVM monitoring threads

✅ **10. Concurrency Utilities (Advanced)**

Class	Purpose
ExecutorService	Manages thread pool
Future	Stores result of async task
Callable<T>	Returns result and can throw exception
Semaphore	Control access to limited resources
CountDownLatch	Wait for set of threads to finish
ReentrantLock	Alternative to synchronized

✅ **11. Asynchronous vs Synchronous**

Feature	Synchronous	Asynchronous
Execution	Tasks wait in sequence	Tasks can run concurrently
Use Case	Simple tasks	I/O, network, parallel tasks
Example	t.run()	t.start() or thread pool

✅ **12. MCQ Tips & Tricky Points**

1. **Which method starts a thread?**
   ✅ start() (not run())
2. **Which interface must be implemented for thread with return value?**
   ✅ Callable<T>
3. **Which method causes thread to pause?**
   ✅ sleep()
4. **Which state is entered after start() but before run()?**
   ✅ RUNNABLE
5. **What happens if run() is called directly?**
   ❌ No new thread created → executes in current thread
6. **Can two threads access the same synchronized method?**
   ❌ Not at the same time on same object
7. **What is thread starvation?**
   ✅ Lower-priority threads never get CPU

✅ **Summary Diagram**

Thread Creation → start() → run() → terminate

↘

race condition → use synchronized

✅ **1. Class Lock in Java**

- A **class lock** is acquired when a synchronized **static method** or block is used.
- It locks the .class object associated with the class (not object-specific).

```
class A {
  public static synchronized void print() { ... } // Class lock
}
```

🔒 All instances share one class-level lock.

🔸 **MCQ Tip:**

What is locked in a static synchronized method?

✅ Class-level lock (ClassName.class)

✅ **2. extends Thread vs implements Runnable**

Feature	extends Thread	implements Runnable
Inheritance	Can't extend any other class	Can extend another class
Flexibility	❌ Less flexible	✅ More flexible
Object Sharing	❌ Not possible	✅ Can share Runnable with many threads
Recommended?	❌ No	✅ Yes

```
// Runnable approach
class Task implements Runnable {
  public void run() {
    System.out.println("Runnable");
  }
}

// Thread approach
class TaskThread extends Thread {
  public void run() {
    System.out.println("Thread");
  }
}
```

🔸 **MCQ Tip:**

Which is preferred: extends Thread or implements Runnable?

✅ implements Runnable (for flexibility and scalability)

✅ 3. Synchronized Method vs Synchronized Block

Feature	synchronized method	synchronized block
Locks entire method	✅ Yes	❌ Only selected code block
Flexibility	❌ Less	✅ High control over lock granularity
Performance	Slower	Faster (only needed section locked)

```
// Synchronized block
synchronized(this) {
   // only this block is locked
}
```

🔸 **MCQ Tip:**

Which is more flexible: synchronized method or block?

✅ Synchronized block

✅ 4. Synchronized Block Details

```
synchronized (lockObject) {
   // critical section
}
```

- Lock can be this (object lock) or a separate object.
- Only one thread can hold the lock at a time.

✅ 5. wait(), notify(), notifyAll()

Defined in java.lang.Object and used for **inter-thread communication** (producer-consumer pattern).

Method	Description
wait()	Tells thread to wait, releases the lock
notify()	Wakes up one waiting thread
notifyAll()	Wakes up all waiting threads

➞ Must be used inside synchronized block or method.

```
synchronized (obj) {
   obj.wait();   // waits
   obj.notify();  // wakes one thread
}
```

🔸 **MCQ Tip:**

Which class defines wait/notify?

✅ Object class

Can wait() be called outside synchronized block?

❌ No → IllegalMonitorStateException

✅ 6. Thread Lifecycle

```
NEW → RUNNABLE → RUNNING → TERMINATED
        ↓
    BLOCKED / WAITING
```

State	Description
NEW	Thread created but not started
RUNNABLE	Ready to run
RUNNING	Actually executing
BLOCKED	Waiting for monitor lock
WAITING	Waiting indefinitely (wait/join)
TERMINATED	Completed or interrupted

✅ **7. Thread-Based Multitasking**

- Java uses **thread-based multitasking** (multiple threads in a single program).
- Threads share same memory, leading to better performance than process-based multitasking.

✅ **8. Process-Based vs Thread-Based Multitasking**

Feature	Process-Based	Thread-Based
Memory	Each process has own memory	Threads share memory
Overhead	High (context switching)	Low
Java Uses	❌	✅ Yes (via Thread API)

🔸 **MCQ Tip:**

Does Java support process-based multitasking directly?

❌ No, it supports thread-based multitasking

✅ **9. Transactions in Java (Intro)**

🔹 A **transaction** is a sequence of operations performed as a **single unit of work**.

If any step fails, the entire transaction rolls back.

- Mainly used in JDBC, JPA, Spring:

```
connection.setAutoCommit(false);
try {
  // multiple DB operations
  connection.commit();
} catch(Exception e) {
  connection.rollback(); // atomicity
}
```

🔸 **Key ACID Properties:**

Property	Description
Atomicity	All-or-nothing
Consistency	Data is valid after transaction
Isolation	Transactions don’t interfere
Durability	Data persists even after crash

✅ **Quick MCQs Summary**

Question	Answer
Preferred threading approach in Java?	implements Runnable
Locks entire method or selected block?	synchronized block
Where is wait()/notify() defined?	Object class
Can wait() be used outside sync block?	❌ No
Class lock is applied on?	ClassName.class object
Java supports which multitasking type?	Thread-based
What ensures thread-safe critical section?	Synchronization

✅ **Executors, ReentrantLock, and Thread Class in Java (Detailed + MCQ Tips)**

🔷 **1. Executors Framework (Java 5+)**

The java.util.concurrent.Executors class provides **thread pool management**, replacing manual thread creation.

✅ **Why Executors?**

- Manages a pool of threads (reuses)
- Improves performance over creating new threads manually
- Helps in handling thousands of concurrent tasks efficiently

---

✅ **Common Methods of Executors**

Method	Description
Executors.newFixedThreadPool(n)	Fixed size thread pool
Executors.newCachedThreadPool()	Creates new threads as needed
Executors.newSingleThreadExecutor()	Only one thread used for all tasks
Executors.newScheduledThreadPool(n)	Supports delayed or periodic tasks

---

✅ **Example**

```
ExecutorService executor = Executors.newFixedThreadPool(2);

Runnable task = () -> System.out.println("Thread: " +
Thread.currentThread().getName());

executor.submit(task);
executor.shutdown();
```

🔸 **MCQ Tip:**

What interface does Executors return?

✅ ExecutorService

Which method shuts down executor immediately?

✅ shutdownNow()

🔷 **2. ReentrantLock (Advanced Locking)**

Available in: java.util.concurrent.locks.ReentrantLock

✅ **Why use ReentrantLock?**

- Offers **explicit locking** with more flexibility than synchronized
- Supports **tryLock()**, **interruptible lock**, and **fairness policy**

✅ **Example:**

```
ReentrantLock lock = new ReentrantLock();

lock.lock();  // acquire
try {
   // critical section
} finally {
   lock.unlock();  // always release in finally
}
```

✅ **Key Methods:**

Method	Description
lock()	Acquires lock
unlock()	Releases lock
tryLock()	Attempts to acquire without blocking
isHeldByCurrentThread()	Checks if current thread holds lock

🔸 **Reentrant Nature:**

- A thread **can acquire the same lock multiple times**
- But must release it **same number of times**

🔸 **MCQ Tip:**

What happens if unlock() is not in finally?

❌ Lock may not be released → Deadlock risk

Is ReentrantLock thread-safe?

✅ Yes

🔷 **3. Thread Class in Detail**

Java's Thread class is in **java.lang package**.

✅ **Key Constructors:**

Thread()
Thread(Runnable target)
Thread(String name)
Thread(Runnable target, String name)

✅ **Common Methods**

Method	Description
start()	Starts thread (calls run())

Method	Description
run()	Actual code to execute
sleep(ms)	Sleep current thread
join()	Wait for thread to die
isAlive()	Checks if thread is alive
setDaemon(true)	Marks thread as daemon thread
getName()	Returns thread name
getId()	Returns thread ID
getPriority()	Thread priority (1 to 10)
interrupt()	Interrupts a sleeping thread

🔸 **Thread Priorities**

Thread.MIN_PRIORITY = 1
Thread.NORM_PRIORITY = 5
Thread.MAX_PRIORITY = 10

🔸 **Daemon Thread**

- Runs in background
- JVM exits when only daemon threads are running

🔸 **MCQ Tips:**

Which method runs in a new thread?
✅ start()
What if run() is called directly instead of start()?
❌ Runs in main thread
Can a thread be restarted?
❌ No, once start() is called, it cannot be restarted
Which thread is garbage collector?
✅ Daemon thread

🔷 **Quick MCQ Recap**

Concept	Question	Answer
Executors	Returns thread pool object	✅ ExecutorService
ReentrantLock	Can one thread acquire it multiple times?	✅ Yes (must unlock same times)
Thread class	What starts new thread?	✅ start()
run() vs start()	What happens if run() called directly?	❌ No new thread
unlock() safety	Where should unlock() be placed?	✅ In finally block
Daemon	When does JVM exit with daemon threads?	✅ When all non-daemon threads finish

🧠 **Flash Summary:**

- **ExecutorService** → Thread pool for managing threads efficiently
- **ReentrantLock** → Explicit, flexible locking with re-entry, tryLock, fairness
- **Thread Class** → Base of Java multithreading; always use start()

✅ Java File Handling + Serialization + Streams + Object Storage

---

✅ 1. File Handling Basics

Java provides file handling via java.io and java.nio packages.

🔹 **Key Classes:**

Class	Purpose
File	Represent file or directory path
FileReader, FileWriter	Reading/Writing character data
FileInputStream, FileOutputStream	Reading/Writing binary data
BufferedReader, BufferedWriter	Buffered character streams

✅ File Example:

```
File f = new File("data.txt");
f.createNewFile();
System.out.println(f.exists()); // true
```

✅ 2. Streams in Java

Java I/O is based on **streams** (flow of data).

Stream Type	Direction	Character or Byte
InputStream	Read	Byte
OutputStream	Write	Byte
Reader	Read	Character
Writer	Write	Character

✅ **Common Classes:**

- **Byte-based**: FileInputStream, FileOutputStream
- **Char-based**: FileReader, FileWriter
- **Buffered**: BufferedReader, BufferedWriter
- **Object**: ObjectInputStream, ObjectOutputStream

✅ 3. Serialization & Deserialization

🔹 **What is Serialization?**

The process of converting a Java object into **byte stream**.
Used to store object in file or transmit over network.

🔹 **What is Deserialization?**

Reconstructing object from a **byte stream**.

✅ **Key Interface:**

```
class Student implements Serializable { ... }
```

- Belongs to java.io.Serializable
- **Marker Interface** (no methods)

✅ **serialVersionUID**

- Unique ID used for version control during deserialization

```
private static final long serialVersionUID = 1L;
```

⚠️ If serialVersionUID mismatches → InvalidClassException

✅ **Default Constructor & Serialization**

- **Constructor is NOT called during deserialization**
- JVM restores object using byte stream

✅ **Storing Object in File**

```
ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("data.ser"));
oos.writeObject(obj);  // serialization
oos.close();
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("data.ser"));
Student s = (Student) ois.readObject();  // deserialization
ois.close();
```

✅ **4. Reading/Writing Multiple Objects**

```
oos.writeObject(obj1);
oos.writeObject(obj2);
Object o1 = ois.readObject();
Object o2 = ois.readObject();
```

✅ **5. Externalizable (Custom Serialization)**

Use when you want **full control** over how object is serialized/deserialized.

```
class Employee implements Externalizable {
  String name;
  int age;

  public void writeExternal(ObjectOutput out) throws IOException {
    out.writeObject(name);
    out.writeInt(age);
  }

  public void readExternal(ObjectInput in) throws IOException,
ClassNotFoundException {
    name = (String) in.readObject();
    age = in.readInt();
  }
}
```

✅ **6. Incompatible/Compatible Serialization**

- **Compatible**: Same class, same serialVersionUID, compatible fields.
- **Incompatible**: Different UID, removed/added fields without default.

🧨 Error: InvalidClassException

✅ **7. Serialization with Inner Class**

- Only **static inner classes** can be serialized directly.

```
static class Inner implements Serializable { ... }
```

- Non-static inner class holds reference to outer → causes issues

---

✅ **8. Custom Serialization**

- Use writeObject() and readObject() methods for customization:

```
private void writeObject(ObjectOutputStream out) throws IOException {
  out.defaultWriteObject();
  out.writeInt(secretValue);
}
```

✅ **9. Exceptions in Serialization**

Exception	When It Occurs
NotSerializableException	Class not implementing Serializable
InvalidClassException	UID mismatch
IOException	File/Stream errors
ClassNotFoundException	Class not found during deserialization

---

✅ **10. 5 Ways to Create Objects in Java**

Method	Example
1. new keyword	Student s = new Student();
2. clone()	Student s2 = (Student) s1.clone();
3. Reflection	Class.forName("Student").newInstance();
4. Constructor.newInstance()	Using Constructor class in Reflection
5. **Deserialization**	ObjectInputStream.readObject()

✅ **MCQ Tips**

✅ **Q1: Which interface is used for serialization?**

➡️ java.io.Serializable

✅ **Q2: Which method stores object in file?**

➡️ writeObject()

✅ **Q3: What happens if UID mismatches?**

➡️ InvalidClassException

✅ **Q4: Can constructor be called in deserialization?**

❌ No

✅ **Q5: Which keyword helps in custom serialization?**

➡️ transient, writeObject(), readObject()

✅ **Q6: Can non-static inner class be serialized?**
❌ No (holds outer reference)

🧠 **Summary Table**

Concept	Notes
Serializable	Marker interface, for object persistence
Externalizable	Complete control over object I/O
serialVersionUID	Used for class version compatibility
writeObject/readObject	Used for custom serialization
Deserialization	Does **not** invoke constructor
transient	Field will **not** be serialized

---

5 Different Ways to Create Objects in Java

There are five total ways to create objects in Java, which are explained below with their examples followed by bytecode of the line which is creating the object.

Using new keyword	} → constructor gets called
Using newInstance() method of Class class	} → constructor gets called
Using newInstance() method of Constructor class	} → constructor gets called
Using clone() method	} → no constructor call
Using deserialization [using Serializable interface]	} → no constructor call

✅ Java File Handling + Serialization + Streams + Object Storage

✅ 1. File Handling Basics

Java provides file handling via java.io and java.nio packages.

🔹 **Key Classes:**

Class	Purpose
File	Represent file or directory path
FileReader, FileWriter	Reading/Writing character data
FileInputStream, FileOutputStream	Reading/Writing binary data
BufferedReader, BufferedWriter	Buffered character streams

✅ File Example:

```
File f = new File("data.txt");
f.createNewFile();
System.out.println(f.exists()); // true
```

✅ 2. Streams in Java

Java I/O is based on **streams** (flow of data).

Stream Type	Direction	Character or Byte
InputStream	Read	Byte
OutputStream	Write	Byte
Reader	Read	Character
Writer	Write	Character

✅ Common Classes:

- **Byte-based**: FileInputStream, FileOutputStream
- **Char-based**: FileReader, FileWriter
- **Buffered**: BufferedReader, BufferedWriter
- **Object**: ObjectInputStream, ObjectOutputStream

✅ 3. Serialization & Deserialization

🔹 **What is Serialization?**

The process of converting a Java object into **byte stream**.
Used to store object in file or transmit over network.

🔹 **What is Deserialization?**

Reconstructing object from a **byte stream**.

✅ Key Interface:

```
class Student implements Serializable { ... }
```

- Belongs to java.io.Serializable
- **Marker Interface** (no methods)

✅ **serialVersionUID**

- Unique ID used for version control during deserialization

```
private static final long serialVersionUID = 1L;
```

⚠️ If serialVersionUID mismatches → InvalidClassException

✅ **Default Constructor & Serialization**

- **Constructor is NOT called during deserialization**
- JVM restores object using byte stream

✅ **Storing Object in File**

```
ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("data.ser"));
oos.writeObject(obj);  // serialization
oos.close();
ObjectInputStream ois = new ObjectInputStream(new FileInputStream("data.ser"));
Student s = (Student) ois.readObject();  // deserialization
ois.close();
```

✅ **4. Reading/Writing Multiple Objects**

```
oos.writeObject(obj1);
oos.writeObject(obj2);
Object o1 = ois.readObject();
Object o2 = ois.readObject();
```

✅ **5. Externalizable (Custom Serialization)**

Use when you want **full control** over how object is serialized/deserialized.

```
class Employee implements Externalizable {
  String name;
  int age;

  public void writeExternal(ObjectOutput out) throws IOException {
    out.writeObject(name);
    out.writeInt(age);
  }

  public void readExternal(ObjectInput in) throws IOException,
ClassNotFoundException {
    name = (String) in.readObject();
    age = in.readInt();
  }
}
```

✅ **6. Incompatible/Compatible Serialization**

- **Compatible**: Same class, same serialVersionUID, compatible fields.
- **Incompatible**: Different UID, removed/added fields without default.

🧨 Error: InvalidClassException

✅ **7. Serialization with Inner Class**

- Only **static inner classes** can be serialized directly.

```
static class Inner implements Serializable { ... }
```

- Non-static inner class holds reference to outer → causes issues

---

✅ **8. Custom Serialization**

- Use writeObject() and readObject() methods for customization:

```
private void writeObject(ObjectOutputStream out) throws IOException {
  out.defaultWriteObject();
  out.writeInt(secretValue);
}
```

✅ **9. Exceptions in Serialization**

Exception	When It Occurs
NotSerializableException	Class not implementing Serializable
InvalidClassException	UID mismatch
IOException	File/Stream errors
ClassNotFoundException	Class not found during deserialization

---

✅ **10. 5 Ways to Create Objects in Java**

Method	Example
1. new keyword	Student s = new Student();
2. clone()	Student s2 = (Student) s1.clone();
3. Reflection	Class.forName("Student").newInstance();
4. Constructor.newInstance()	Using Constructor class in Reflection
5. **Deserialization**	ObjectInputStream.readObject()

✅ **MCQ Tips**

✅ **Q1: Which interface is used for serialization?**

➡️ java.io.Serializable

✅ **Q2: Which method stores object in file?**

➡️ writeObject()

✅ **Q3: What happens if UID mismatches?**

➡️ InvalidClassException

✅ **Q4: Can constructor be called in deserialization?**

❌ No

✅ **Q5: Which keyword helps in custom serialization?**

➡️ transient, writeObject(), readObject()

✅ **Q6: Can non-static inner class be serialized?**

❌ No (holds outer reference)

🧠 Summary Table

Concept	Notes
Serializable	Marker interface, for object persistence
Externalizable	Complete control over object I/O
serialVersionUID	Used for class version compatibility
writeObject/readObject	Used for custom serialization
Deserialization	Does **not** invoke constructor
transient	Field will **not** be serialized

✅ Is Object class Serializable?

❌ No

🔹 Why?

If Object was Serializable, **every class** in Java would become serializable by default — including **Thread, Socket, DB connections**, etc., which:

- Depend on **JVM and OS state**
- Can't be restored across different JVMs
- Serialization doesn't make sense for these runtime-bound classes

---

✅ What if Object was Serializable?

- Every class would be serializable by default
- Runtime-sensitive classes (Thread, Socket, DB connections) could be incorrectly serialized
- Breaks JVM abstraction → unpredictable behavior

✅ What Gets Written During Serialization?

🔹 1. Class Metadata

- Includes class name and serialVersionUID
- From subclass → superclass (until Object)

🔹 2. Instance Data

- Values of **non-transient**, **non-static** fields
- Written from **superclass → subclass**

🔹 3. "Has-a" References

- Metadata and data of referenced serializable objects (deep tree)
- Skips non-serializable members

✅ Deserialization Flow (Internal Working)

1. Object Read from stream
2. JVM checks class info from serialized object
3. JVM tries to load the class
    - If fails → ClassNotFoundException
4. Object created without calling constructor
5. If superclass is not Serializable:
    - Its constructor is called (and its parents too)
6. Instance fields restored (non-transient only)

🔹 transient fields → default values (0, false, null)

🧠 **MCQ TIPS**

Question	Answer
Is Object class Serializable?	❌ No
What if Object were Serializable?	All Java classes would be serializable
Are constructors called during deserialization?	❌ No (unless superclass is not serializable)
Are transient fields serialized?	❌ No
Does deserialization call new constructor?	❌ No, uses special internal process
What exception if class not found during deserialization?	ClassNotFoundException

---

📌 **1. What is Collection Framework?**

- Part of java.util package
- Provides architecture to store and manipulate **group of objects**
- **Stores object references** (not primitives directly)
- Root interface: Collection<E>

✅ **Interfaces in Collection API**

Interface	Description
Collection	Root of List, Set, Queue
List	Ordered, duplicates allowed
Set	No duplicates
Queue	For FIFO structures
Map	Key-value pair, no duplicate keys

✅ **2. List Implementations**

Class	Features
ArrayList	Resizable array, random access O(1), slow insert/delete
LinkedList	Doubly linked list, fast insert/delete, no random access
Vector	Legacy, thread-safe
Stack	Legacy, LIFO stack built on Vector

🔸 **MCQ Tip:**

Which list gives O(1) access by index?

✅ ArrayList

✅ 3. Set Implementations

Class	Features
HashSet	No duplicates, no order, uses HashMap internally
LinkedHashSet	Maintains insertion order
TreeSet	Sorted, no nulls, uses Red-Black Tree

❗ **TreeSet does not allow heterogeneous elements**

Throws: ClassCastException

✅ 4. Map Implementations

Class	Features
HashMap	Unordered, one null key allowed, not thread-safe
LinkedHashMap	Maintains insertion order
TreeMap	Sorted keys, uses Red-Black tree, no null key
Hashtable	Legacy, thread-safe, no null key/value allowed
ConcurrentHashMap	Thread-safe, better performance than Hashtable

---

🔸 **MCQ Tip:**

Which map is synchronized by default?

✅ Hashtable

Which map maintains key order?

✅ TreeMap

✅ 5. Comparable vs Comparator

Feature	Comparable	Comparator
Interface in	java.lang	java.util
Method	compareTo(Object o)	compare(Object o1, Object o2)
Used for	Natural ordering (default)	Custom sorting
Affects class?	Yes (must implement interface)	No (can be used externally)

🔸 **MCQ Tip:**

Which interface allows custom sort without modifying class?

✅ Comparator

✅ 6. Java Generics

- Introduced in Java 5
- Enables **type safety** at compile time
- Prevents ClassCastException

```
List<String> list = new ArrayList<>();
```

✅ Generic Class

```
class Box<T> {
  T value;
}
```

✅ **Wildcards**

Syntax	Meaning
<?>	Unknown type
<? extends T>	T or subclass of T (read-only)
<? super T>	T or superclass of T (write-safe)

✅ **7. Synchronized Collections**

Option	Thread-Safe?	Example
Collections.synchronizedList()	✅ Yes	Wraps any list to be safe
Vector, Hashtable	✅ Yes	Legacy thread-safe classes
ArrayList, HashMap	❌ No	Use Collections.synchronizedXXX()

🔸 **Example**

```
List<String> list = Collections.synchronizedList(new ArrayList<>());
```

✅ **8. TreeMap Details**

- Sorted map (by natural/comparator order of keys)
- No null key allowed
- Internally uses **Red-Black Tree**

🔸 **TreeSet & TreeMap Rules**

- Must have **comparable** or **comparator**
- No **heterogeneous elements**
- No **null keys** (TreeMap)

✅ **9. Algorithms in Collections API**

In java.util.Collections class:

Method	Purpose
sort(List)	Sorts elements
reverse(List)	Reverses order
shuffle(List)	Randomly shuffles
binarySearch(List, key)	Returns index (sorted list only)
max(), min()	Finds max/min
frequency()	Counts occurrences
replaceAll()	Replaces all with given val

🔸 **MCQ Tip:**

Which class provides utility algorithms like sort()?

✅ java.util.Collections

✅ 10. Collection vs Collections

Feature	Collection	Collections
Type	Interface	Utility class
Part of	java.util	java.util
Purpose	Root for Set/List/Queue	Utility functions (sort, reverse, etc.)

✅ MCQ Cheat Sheet

Question	Answer
Which collection allows duplicates?	List
Which collection maintains insertion order?	LinkedHashMap
Which Set does sorting?	TreeSet
Can TreeSet have null?	❌ No
What happens with heterogeneous objects in TreeSet?	ClassCastException
Is HashSet ordered?	❌ No
Which interface defines compareTo()?	Comparable
Can we sort without modifying class?	✅ Comparator
Which utility class provides sorting, reverse?	Collections
Does HashMap allow null key?	✅ Yes
Which map is thread-safe?	Hashtable

✅ Java Socket Programming + RMI (Remote Method Invocation)

📌 1. What is Socket Programming?

Java **Socket Programming** enables communication between machines over a network using **TCP/IP** protocols.

✅ Packages and Core Classes

Package	Description
java.net	Core socket, server-socket, URL
java.io	For reading/writing stream data

✅ Important Classes & Interfaces

Class / Interface	Description
Socket	Client-side TCP socket
ServerSocket	Listens for client requests
InetAddress	Represents IP address
DatagramSocket	UDP communication
DatagramPacket	UDP packet
URL, URLConnection	HTTP-based communication

✅ **TCP Communication (Reliable)**

🔹 **Server Side Code**

```
ServerSocket server = new ServerSocket(5000);
Socket socket = server.accept();
InputStream in = socket.getInputStream();
OutputStream out = socket.getOutputStream();
```

🔹 **Client Side Code**

```
Socket client = new Socket("localhost", 5000);
OutputStream out = client.getOutputStream();
InputStream in = client.getInputStream();
```

✅ **UDP Communication (Fast, Unreliable)**

```
DatagramSocket ds = new DatagramSocket();
byte[] data = "Hello".getBytes();
InetAddress ip = InetAddress.getByName("localhost");
DatagramPacket dp = new DatagramPacket(data, data.length, ip, 5000);
ds.send(dp);
```

✅ **Key Methods (TCP)**

Class	Method	Purpose
Socket	getInputStream()	For reading from socket
	getOutputStream()	For writing to socket
ServerSocket	accept()	Waits for client connection
	close()	Closes connection
InetAddress	getByName(), getLocalHost()	IP Resolution

✅ **MCQ TIPS - Socket**

Q: Which class is used by TCP server?

✅ ServerSocket

Q: Which class sends data via UDP?

✅ DatagramSocket

Q: Which method listens for a connection?

✅ accept()

Q: Which protocol is reliable?

✅ TCP

Q: What does Socket.getInputStream() return?

✅ InputStream

✅ **2. Java RMI (Remote Method Invocation)**

🔹 **What is RMI?**

- Java's way to **invoke methods remotely** across JVMs
- Uses **stub & skeleton architecture**
- Runs over TCP/IP

✅ RMI Architecture Components

Component	Description
Client	Calls remote method
Server	Implements remote interface
Stub (Proxy)	Client-side helper to connect to server
Skeleton (removed in Java 1.2+)	Server-side dispatch logic (internal)
RMI Registry	Like DNS – registers and looks up objects

✅ Core RMI Packages & Classes

Package	Purpose
java.rmi	Core RMI interfaces
java.rmi.server	Server-side skeleton/stub mgmt
java.rmi.registry	RMI registry utility

✅ Key Interfaces

Interface	Purpose
Remote	Marker interface – indicates remote use
Serializable	Transfer objects over network
UnicastRemoteObject	Used to export remote objects

✅ RMI Steps

1. **Create remote interface**

```
public interface MyRemote extends Remote {
  String sayHello() throws RemoteException;
}
```

2. **Implement interface on server**

```
public class MyRemoteImpl extends UnicastRemoteObject implements MyRemote {
  public String sayHello() {
    return "Hello from server";
  }
}
```

3. **Register object with RMI Registry**

```
LocateRegistry.createRegistry(1099);
Naming.rebind("HelloService", new MyRemoteImpl());
```

4. **Client lookup**

```
MyRemote stub = (MyRemote) Naming.lookup("rmi://localhost/HelloService");
System.out.println(stub.sayHello());
```

✅ RMI Exceptions

Exception	Reason
RemoteException	Any remote failure
NotBoundException	Lookup name not found in registry

Exception	Reason
MalformedURLException	Improper RMI URL

✅ **MCQ TIPS - RMI**

Q: Which interface must be implemented to define remote object?

✅ Remote

Q: Which class is used to export remote object?

✅ UnicastRemoteObject

Q: Which package has RMI registry?

✅ java.rmi.registry

Q: What protocol does RMI use?

✅ TCP/IP

Q: Which exception is thrown if object not found in registry?

✅ NotBoundException

✅ **Key Differences: Socket vs RMI**

Feature	Socket Programming	RMI
Comm Type	Low-level (streams, bytes)	High-level (method calls)
Protocol	TCP / UDP	TCP
Flexibility	More control	More abstraction
Use-case	Custom protocol / cross-lang	Java-to-Java remote invocation
Security	Manual	Built-in (SecurityManager, policy)

🌐 **Java Internationalization (i18n) & Localization (L10n)**

✅ **1. What is Internationalization (i18n)?**

- Process of **designing a Java application** so that it **can be adapted to different languages, regions, or cultures** without engineering changes.
- The term **i18n** comes from:
  i + 18 letters + n = internationalization

✨ **Purpose:**

- **Separate** locale-specific data (like language, date formats, currency) from source code.
- Makes applications ready for **global audiences**.

✅ **2. What is Localization (L10n)?**

- Actual **adapting** of the internationalized application to a specific **locale** (language + country).
- L10n = l + 10 letters + n = localization

✨ **Example:**

- Translating "Hello" to:
  - "Bonjour" for French (fr_FR)
  - "नमस्ते" for Hindi (hi_IN)

✅ 3. Key Java Packages

Package	Purpose
java.util.Locale	Encapsulates language & region
java.util.ResourceBundle	Loads localized resources
java.text.*	Formatting of date, number, messages

✅ 4. Locale Class

Represents a specific **geographical**, **political**, or **cultural** region.

```
Locale locale = new Locale("fr", "FR"); // French - France
```

Common Constructors:

```
new Locale(language)
new Locale(language, country)
```

Common Methods:

```
getLanguage() → "fr"
getCountry()  → "FR"
toString()    → "fr_FR"
```

✅ 5. ResourceBundle

Loads language-specific **.properties** files based on the Locale.

🔹 **Example file names:**

File Name	Locale
Messages.properties	default
Messages_en.properties	English
Messages_hi_IN.properties	Hindi (India)

🔹 **Example content:**

Messages_hi_IN.properties

```
greeting=नमस्ते
farewell=अलविदा
```

🔹 **Java Code Example:**

```
import java.util.*;

public class TestI18n {
   public static void main(String[] args) {
      Locale locale = new Locale("hi", "IN");
      ResourceBundle bundle = ResourceBundle.getBundle("Messages", locale);

      System.out.println(bundle.getString("greeting"));  // नमस्ते
      System.out.println(bundle.getString("farewell"));  // अलविदा
   }
}
```

🔸 The JVM will automatically pick the most appropriate file based on the Locale.

✅ 6. Formatting (Numbers, Currency, Date)

Java provides **locale-aware formatters** in java.text package.

🔹 NumberFormat

```
NumberFormat nf = NumberFormat.getCurrencyInstance(new Locale("en", "US"));
System.out.println(nf.format(1234.56)); // $1,234.56
```

🔹 DateFormat

```
DateFormat df = DateFormat.getDateInstance(DateFormat.FULL, new Locale("fr", "FR"));
System.out.println(df.format(new Date())); // mercredi 7 juillet 2025
```

✅ 7. Java 8+: MessageFormat for Placeholder Text

```
String pattern = "{0}, you have {1} new messages";
MessageFormat mf = new MessageFormat(pattern, Locale.US);
System.out.println(mf.format(new Object[]{"John", 5}));
```

✅ 8. Best Practices

- ✅ Always use ResourceBundle instead of hardcoding
- ✅ Keep default .properties file (fallback)
- ✅ Avoid using static strings in UI/console
- ✅ Group UI strings in one file, error messages in another
- ✅ Use tools like gettext, poedit for big projects

📌 MCQ TIPS

Question	Answer
What does i18n stand for?	Internationalization
What does Locale represent?	Language + Region
Which class is used to fetch locale resources?	ResourceBundle
What if file Messages_hi_IN.properties not found?	Fallback to default
Which package is used for formatting?	java.text
Which method returns currency for a locale?	NumberFormat.getCurrencyInstance()
Can we hardcode locale-specific values?	❌ No – use ResourceBundle

---

✅ Summary

Term	Meaning
i18n	Prepare app for multiple languages
L10n	Adapt app to a specific language
Locale	Represents language + country
ResourceBundle	Loads key-value text files
NumberFormat	Formats currency/number per locale
DateFormat	Formats date/time

✅ **What is Stream API?**

- Introduced in **Java 8** (java.util.stream)
- **Processes collections in a functional style**
- Supports **pipeline operations**:
  - **Intermediate ops** (e.g. map(), filter(), peek())
  - **Terminal ops** (e.g. forEach(), collect(), reduce())

🔹 **Key Features:**

- Declarative (not imperative)
- Supports **lazy evaluation**
- Works on **Stream of data**, not modifying source
- Allows **parallel operations**

✅ **Stream API vs Collections API**

Feature	Collections API	Stream API
Stores data?	✅ Yes	❌ No (Processes data only)
Supports iteration?	✅ Yes (external)	✅ Yes (internal)
Parallel processing?	❌ Manual	✅ Easy via .parallelStream()
Memory efficient?	❌ No (holds entire collection)	✅ Yes (lazy, pipeline)
Modifies source?	✅ Yes	❌ No (returns new Stream)

✅ **Intermediate Operations (Lazy)**

These return another stream and are **not executed immediately**.

Method	Purpose
map()	Transforms each element
filter()	Filters elements
peek()	Debug/inspect elements
sorted()	Sorts elements
distinct()	Removes duplicates
flatMap()	Flattens nested structure
limit()	Truncates stream to given size
skip()	Skips first n elements

🔸 **map() vs peek()**

```
list.stream().map(x -> x * 2);  // transforms
list.stream().peek(System.out::println); // debug view
```

- map() transforms and returns a new stream.
- peek() is for **debugging** (non-terminal); use before collect or forEach.

✅ **Terminal Operations (Eager)**

These **trigger** the stream pipeline.

Method	Description
forEach()	Perform action on each element
collect()	Reduces stream into collection or value
reduce()	Combines elements into single result
count()	Counts elements
anyMatch()	Returns true if any element matches predicate
allMatch()	True if all match
noneMatch()	True if none match
findFirst()	Returns first element
findAny()	Returns any element (esp. in parallel)

✅ **Stream API Example**

```
List<String> names = List.of("Alice", "Bob", "Charlie");

List<String> result = names.stream()
  .filter(n -> n.startsWith("A"))
  .map(String::toUpperCase)
  .collect(Collectors.toList());

System.out.println(result); // [ALICE]
```

✅ **reduce() Explained**

```
int sum = List.of(1, 2, 3).stream().reduce(0, (a, b) -> a + b);
// sum = 6
```

Reduce Form	Use Case
reduce(identity, accumulator)	Accumulate with initial
reduce(accumulator)	Returns Optional

✅ **MCQ TIPS**

Question	Answer
Is Stream a data structure?	❌ No
Does Stream API store data?	❌ No
Which is lazy: map or collect?	✅ map
Does forEach() return a Stream?	❌ No (it's terminal)
Which method collects results to a list?	✅ collect(Collectors.toList())
Difference between map() and flatMap()	map transforms; flatMap flattens
Can you reuse a stream?	❌ No (Streams are consumed)
Does reduce() return a stream?	❌ No (returns result)
Which is used for debugging?	✅ peek()

✅ **Quick Code Recap**

```
List<String> list = List.of("a", "b", "c");

list.stream()
  .peek(System.out::println)
  .map(String::toUpperCase)
  .forEach(System.out::println);

// Output: a A, b B, c C
```

✅ **Bonus: Parallel Stream**

```
list.parallelStream().forEach(...); // uses fork-join pool
```

Use only when you have **large datasets** and **independent tasks**.

✅ **Java NIO (New Input/Output)**

📦 **Package:**

```
java.nio.*
java.nio.channels.*
java.nio.file.*
java.nio.file.attribute.*
java.nio.charset.*
```

📌 **1. What is NIO?**

NIO stands for **Non-blocking Input/Output** — designed for scalable and high-performance I/O operations.

It offers:

- **Buffers** instead of streams
- **Channels** instead of InputStream/OutputStream
- **Selectors** for non-blocking I/O
- **Path, Files, FileSystem API** (Java 7+)

✅ **Major Differences: IO vs NIO**

Feature	IO (Old)	NIO (New)
Data Handling	Stream-oriented	Buffer-oriented
Blocking	Always blocking	Supports non-blocking
Multithreading	Thread-per-connection	Single-thread for many channels
Performance	Lower	Higher, especially for large files
New file support	No	Yes (Path, Files APIs)

✅ **2. Buffer (java.nio)**

- A **container** for data that is read/written.
- Types:
  - ByteBuffer, CharBuffer, IntBuffer, etc.

🔹 **Key Buffer Methods:**

```
allocate(int capacity)   // allocates buffer
put() / get()            // write/read from buffer
```

```
flip()              // switch from write to read mode
rewind()            // read again from start
clear()             // resets position to 0
ByteBuffer buffer = ByteBuffer.allocate(10);
buffer.put((byte)65); // add 'A'
buffer.flip();      // switch to read mode
System.out.println(buffer.get()); // 65
```

✅ **3. Channel (java.nio.channels)**

- Like a stream but **bi-directional**
- Can read and write using **buffers**
- Works with **FileChannel**, **SocketChannel**, etc.

```
RandomAccessFile file = new RandomAccessFile("data.txt", "rw");
FileChannel channel = file.getChannel();

ByteBuffer buffer = ByteBuffer.allocate(1024);
channel.read(buffer);    // Read from file into buffer
```

🔹 **Common Channel Classes:**

- FileChannel → File I/O
- SocketChannel, ServerSocketChannel → Network I/O
- DatagramChannel → UDP
- Channels.newChannel(InputStream) → Wrapping IO into Channel

---

✅ **4. Selector (java.nio.channels.Selector)**

- **Single thread** can monitor multiple channels (used in servers)
- Good for **multiplexed non-blocking I/O**
- Works with SelectableChannel (like SocketChannel)

```
Selector selector = Selector.open();
channel.register(selector, SelectionKey.OP_READ);
```

✅ **5. Path & Files API (Java 7+)**

Package: java.nio.file.*

🔹 **Path interface**

- Represents a file or directory path.
- Created using Paths.get(...)

```
Path path = Paths.get("test.txt");
System.out.println(path.getFileName());
```

🔹 **Files utility class**

- Used for file manipulation (read, write, copy, delete)

```
Files.copy(sourcePath, targetPath);
Files.delete(path);
Files.write(path, List.of("Hello"), StandardCharsets.UTF_8);
```

✅ **6. WatchService (Directory Watcher)**

- Monitors directory for **file events** (create, delete, modify)

```
WatchService watcher = FileSystems.getDefault().newWatchService();
```

path.register(watcher, StandardWatchEventKinds.ENTRY_CREATE);

✅ 7. Character Encoding (Charset)

Package: java.nio.charset

```
Charset charset = StandardCharsets.UTF_8;
byte[] bytes = "Hello".getBytes(charset);
String str = new String(bytes, charset);
```

✅ MCQ TIPS

Question	Answer
What is the core data unit in NIO?	Buffer
What class allows random access to files?	FileChannel
Which package contains Path and Files classes?	java.nio.file
Which method switches buffer from write to read mode?	flip()
What allows single-threaded multiple channel processing?	Selector
Which method registers a channel with selector?	register()
Can a channel read and write both ways?	✅ Yes
Which method gets bytes from a string in NIO?	getBytes(Charset)
What is WatchService used for?	Monitoring file system

✅ Summary

Concept	Core Class	Purpose
Buffer	ByteBuffer, etc.	Read/write container for data
Channel	FileChannel, etc.	Bi-directional I/O communication
Path	Path	Represents file path
Files	Files	File utility methods
Selector	Selector	Non-blocking multi-channel monitor
WatchService	WatchService	File system change listener
Charset	Charset	Character encoding/decoding

✅ Working of Channel, Buffer, Path vs Paths, and Asynchronous File I/O using CompletionHandler

---

📌 1. Working of Channel and Buffer (Core of Java NIO)

In Java NIO:

- Data is **not read/written directly** — instead, it's done via a **Buffer**.
- A **Channel** is used to transfer data **between source (like file/socket)** and **buffer**.

🔁 **Flow:**

[File/Socket] ⇄ Channel ⇄ Buffer ⇄ Program

🔹 **How it works:**

```
// Reading data from file using channel & buffer
FileInputStream fis = new FileInputStream("test.txt");
FileChannel channel = fis.getChannel();

ByteBuffer buffer = ByteBuffer.allocate(1024);  // 1 KB buffer

int bytesRead = channel.read(buffer); // Reads into buffer
buffer.flip(); // Switch to read mode

while(buffer.hasRemaining()) {
   System.out.print((char) buffer.get()); // Read data byte by byte
}

channel.close();
```

🔸 **Key Methods of Buffer**

Method	Use
allocate()	Create buffer
put()	Write to buffer
get()	Read from buffer
flip()	Switch from write to read mode
clear()	Reset buffer for next write
hasRemaining()	Check if buffer has unread data

📌 2. Path vs Paths

Path	Paths
Interface	Utility class to get Path objects
Represents path	Contains static factory methods
From: java.nio.file	From: java.nio.file

🔹 **Example:**

```
Path path = Paths.get("data.txt"); // Factory method from Paths
System.out.println(path.getFileName()); // "data.txt"
```

📌 **3. Asynchronous File I/O in Java NIO.2 (Java 7+)**

For non-blocking file operations using **callback**, use:
java.nio.channels.AsynchronousFileChannel

➕ **Benefits:**

- Useful in **server-side** or **UI-based applications**
- Avoids blocking main thread
- Uses **CompletionHandler** or **Future**

✅ **Example with CompletionHandler**

```
Path path = Paths.get("example.txt");

AsynchronousFileChannel asyncChannel =
  AsynchronousFileChannel.open(path, StandardOpenOption.READ);

ByteBuffer buffer = ByteBuffer.allocate(1024);

asyncChannel.read(buffer, 0, buffer, new CompletionHandler<Integer, ByteBuffer>() {
  @Override
  public void completed(Integer result, ByteBuffer attachment) {
    System.out.println("Read completed: " + result + " bytes");
    attachment.flip();
    while (attachment.hasRemaining()) {
      System.out.print((char) attachment.get());
    }
  }

  @Override
  public void failed(Throwable exc, ByteBuffer attachment) {
    System.err.println("Read failed: " + exc.getMessage());
  }
});
```

- read() is non-blocking.
- CompletionHandler runs on completion or failure.
- attachment = buffer passed to handler (to read data from).

✅ **Writing Asynchronously**

```
AsynchronousFileChannel asyncChannel =
  AsynchronousFileChannel.open(path, StandardOpenOption.WRITE,
StandardOpenOption.CREATE);

ByteBuffer buffer = ByteBuffer.wrap("Hello Async!".getBytes());

asyncChannel.write(buffer, 0, buffer, new CompletionHandler<Integer, ByteBuffer>() {
  @Override
  public void completed(Integer result, ByteBuffer attachment) {
    System.out.println("Write completed: " + result + " bytes");
```

```
    }

    @Override
    public void failed(Throwable exc, ByteBuffer attachment) {
        System.err.println("Write failed: " + exc.getMessage());
    }
});
```

🧠 **MCQ TIPS**

Question	Answer
Which class represents file path in NIO?	Path
Which class provides static methods for path creation?	Paths
What connects buffer to I/O in NIO?	Channel
Which method switches buffer to read mode?	flip()
What allows non-blocking file I/O with callback?	AsynchronousFileChannel
Which interface handles async callback?	CompletionHandler
Which buffer is preferred for text data?	CharBuffer
What is used to allocate a buffer?	ByteBuffer.allocate(int)

✅ **Summary**

Concept	Key Class/Method
File I/O (non-blocking)	AsynchronousFileChannel
Read Callback	CompletionHandler<Integer, ByteBuffer>
Buffer for data	ByteBuffer, CharBuffer
File path representation	Path
Path creation	Paths.get(...)
Flip mode (write → read)	flip()

✅ **What is Optional in Java?**

- Introduced in **Java 8**, Optional<T> is a **container object** used to **avoid null references** and prevent NullPointerException.
- It may or may not **contain a non-null value**.

📦 **Package:**

java.util.Optional

✅ **Why Optional?**

Before Optional:

```
String name = user.getName(); // Might be null → NPE
```

With Optional:

```
Optional<String> name = Optional.ofNullable(user.getName());
name.ifPresent(System.out::println);
```

✅ Types of Optional Creation

Method	Purpose	Throws NPE?
Optional.of(T val)	Creates Optional with non-null value	✅ Yes
Optional.empty()	Creates an empty Optional	❌ No
Optional.ofNullable(T val)	Creates Optional that can be null	❌ No

---

🔹 **Examples:**

```
Optional<String> a = Optional.of("hello");
Optional<String> b = Optional.ofNullable(null);
Optional<String> c = Optional.empty();
```

✅ Common Optional Methods

Method	Description
isPresent()	Returns true if value is present
ifPresent(Consumer)	Executes action if value is present
get()	Returns value if present, else throws exception
orElse(default)	Returns value or default
orElseGet(Supplier)	Lazy default computation
orElseThrow()	Throws NoSuchElementException if empty
map(Function)	Transforms value inside Optional
flatMap()	Used when nested Optionals (avoid Optional)
filter(Predicate)	Returns Optional if value matches predicate

---

🔹 **Example Usage**

```
Optional<String> name = Optional.of("Yash");

System.out.println(name.isPresent()); // true
System.out.println(name.get());      // "Yash"
System.out.println(name.orElse("default")); // "Yash"

Optional<String> empty = Optional.empty();
System.out.println(empty.orElse("Default")); // Default
```

🔹 **orElse() vs orElseGet()**

```
String result = optional.orElse("Fallback"); // Always evaluates fallback
String result2 = optional.orElseGet(() -> computeExpensiveDefault()); // Lazy
```

🧠 **MCQ Tip**: orElse() always evaluates the argument, orElseGet() evaluates only if needed.

🔹 **map() vs flatMap()**

```
Optional<String> opt = Optional.of("hello");
Optional<Integer> len = opt.map(String::length); // Optional[5]
```

When nested Optionals:

```
Optional<Optional<String>> nested = Optional.of(Optional.of("data"));
Optional<String> flat = nested.flatMap(e -> e); // Optional["data"]
```

🔹 **filter() Example**

```
Optional<String> email = Optional.of("test@gmail.com");

email.filter(e -> e.endsWith("@gmail.com"))
    .ifPresent(System.out::println); // prints email
```

❗ **Best Practices**

✅ Prefer Optional as **return type**, not constructor or field
✅ Don't use get() without isPresent() or fallback
✅ Use map() and flatMap() to chain safely
✅ Avoid null inside Optional (ex: Optional.of(null) → NPE)

🔥 **MCQ Tips**

Question	Answer
Can Optional.of(null) throw NPE?	✅ Yes
What does Optional.empty() return?	An empty Optional
What method returns default value lazily?	orElseGet()
What does Optional.get() do if no value present?	Throws NoSuchElementException
Which method is used to transform Optional's value?	map()
Which method avoids Optional<Optional> nesting?	flatMap()
Can you use Optional for class fields?	❌ No (not recommended)
Does filter() return Optional if predicate fails?	✅ Yes (Optional.empty())

✅ **Summary Table**

Feature	Explanation
Optional<T>	Wrapper to represent optional value
of()	For non-null value
ofNullable()	Accepts null
empty()	Returns empty Optional
get()	Unsafe — may throw exception
orElse()	Returns fallback value (eager)
orElseGet()	Returns fallback from supplier (lazy)
map()	Transform value
flatMap()	Flatten nested Optional
filter()	Filters based on condition

💡 **Real-world use:**

```
public Optional<User> findUserById(int id) {
  return userRepo.stream()
     .filter(u -> u.getId() == id)
     .findFirst();
}
```

Here's a **deep dive into advanced Optional<T> usage**, focused on:

- isPresent()
- orElse() / orElseGet() / orElseThrow()
- ofNullable()
- filter()
- map() / flatMap()
- Future-ready usage patterns
-
    - MCQ tips and interview-level tricks

✅ **Purpose of Optional**

Optional helps to **write null-safe code** by avoiding NullPointerException.

Instead of:

```
if (user != null && user.getName() != null) {
  System.out.println(user.getName());
}
```

We write:

```
Optional.ofNullable(user)
     .map(User::getName)
     .ifPresent(System.out::println);
```

✅ **1. Optional.ofNullable()**

- Safely wrap a value **which may be null**

```
String name = null;
Optional<String> opt = Optional.ofNullable(name); // ✅ SAFE
```

🔴 Avoid this:

```
Optional.of(null); // ❌ Throws NullPointerException
```

✅ **2. isPresent() and ifPresent()**

- isPresent() → returns true if a value exists
- ifPresent() → executes lambda if present

```
Optional<String> opt = Optional.of("Java");

if (opt.isPresent()) {
  System.out.println(opt.get());
}

opt.ifPresent(val -> System.out.println("Value: " + val));
```

✅ **3. orElse(), orElseGet(), orElseThrow()**

🔹 **orElse(): returns fallback value (eager)**

```
String val = Optional.ofNullable(null).orElse("Default");
```

🔴 Even if value is present, orElse() **evaluates the fallback expression**.

🔹 **orElseGet(): lazy fallback**

```
String val = Optional.ofNullable(null).orElseGet(() -> "LazyDefault");
```

🔹 orElseThrow(): throws exception if empty

```
String val = Optional.ofNullable(null).orElseThrow(() -> new
IllegalArgumentException("Value missing"));
```

✅ **4. filter(Predicate)**

- Only returns non-empty Optional **if condition matches**

```
Optional<String> email = Optional.of("yash@gmail.com");

email.filter(e -> e.endsWith("@gmail.com"))
    .ifPresent(System.out::println);
```

If predicate fails → Optional.empty() is returned.

✅ **5. map() vs flatMap()**

🔹 **map():**

Transforms the value inside Optional.

```
Optional<String> name = Optional.of("yash");
Optional<Integer> len = name.map(String::length); // Optional[4]
```

🔹 **flatMap():**

Use when the mapper returns another Optional<T>.

```
Optional<User> userOpt = Optional.of(new User());
Optional<String> name = userOpt.flatMap(User::getOptionalName); // avoids
Optional<Optional<>>
```

✅ **Real-World Scenario with Optional**

```
public Optional<User> getUserByEmail(String email) {
   return userList.stream()
      .filter(u -> u.getEmail().equals(email))
      .findFirst();
}
Then safely use:
Optional<User> user = getUserByEmail("yash@gmail.com");

String name = user
   .map(User::getName)
   .orElse("Guest");
```

🧠 MCQ Tips

Concept	Tip/Answer
Optional.of(null)	❌ Throws NPE
Best way to wrap null?	✅ Optional.ofNullable()
orElse vs orElseGet()	orElse is **eager**, orElseGet is **lazy**
get() on empty Optional?	❌ Throws NoSuchElementException
Safely access nested objects?	✅ Use map() / flatMap()
filter() behavior?	Returns Optional only if predicate is true

---

✅ Optional Use-Cases in Future Projects

Use Case	Optional Usage
Return values from Repository	Optional<User> findById(int id);
Avoid null checks	Use map(), filter(), orElse()
Optional as method param	❌ Not recommended (use overloading or null check)
REST APIs	Return Optional.empty() for 404 cases
Chaining objects	user.map(User::getAddress).map(Address::getCity)

✅ Code Pattern Summary

```
Optional.ofNullable(user)
    .map(User::getName)
    .filter(n -> !n.isEmpty())
    .orElse("No name");
```

✅ Interview Questions

1. **When to use Optional?**
    - As method return type when result may be null.
2. **When not to use Optional?**
    - For fields or method parameters (adds overhead).
3. **Difference between map() and flatMap()?**
    - map() wraps result in Optional.
    - flatMap() unwraps nested Optional.
4. **Lazy fallback in Optional?**
    - orElseGet() is lazy.
